

UNIVERSITÀ DEGLI STUDI DI NAPOLI
“FEDERICO II”

FACOLTÀ DI SCIENZE MATEMATICHE, FISICHE E NATURALI



TESI DI LAUREA IN FISICA

*Studio di un controllo senso-motorio ispirato al
biologico realizzato tramite reti CTRNN*

Relatori:

Prof. R. Prevete

Prof. G. Trautteur

Correlatore:

Prof. E. Burattini

Candidato:

Francesco Donnarumma

Matricola 60/211

ANNO ACCADEMICO 2005/2006

Indice

1	Introduzione	5
1.1	Obiettivi	5
1.2	Capitoli	9
2	Modello di sistema senso-motorio	11
2.1	Sistemi di controllo: ispirazione biologica	11
2.1.1	Sistema motorio corticale	12
2.1.2	Corteccia Agranulare Frontale	16
2.1.2.1	Corteccia premotoria mesiale	18
2.1.2.2	Corteccia premotoria dorsale	19
2.1.2.3	Corteccia premotoria ventrale	20
2.1.3	Corteccia parietale posteriore	22
2.1.3.1	Lobo parietale inferiore	22
2.2	Circuiti Parieto-premotori	24
2.3	Circuito AIP-F5	26
2.3.1	Modello a blocchi	31
2.4	Codifica del piano: sequenza di primitive	37
2.4.1	Reti di Tani	37
2.4.2	Gerarchia di Reti	38

<i>INDICE</i>	3
3 Reti Neurali a tempo continuo	41
3.1 Neurone biologico	41
3.1.1 Dinamica dei neuroni e sinapsi	45
3.2 Modello fisico di neurone	46
3.3 Modello di rete: CTRNN	49
3.3.1 Legge di aggiornamento	50
3.4 Rete CTRNN come Sistema di Controllo	52
3.5 Rete CTRNN come Sistema Dinamico	53
3.5.1 CTRNN come sistema di ODE	54
3.5.2 Sistema di ODE come sistema Dinamico . .	55
3.5.3 Stabilità	56
3.5.4 Punti fissi	57
3.5.5 Comportamenti al limite e regolarità . . .	58
3.5.6 Biforcazioni	60
3.6 Capacità di memoria di una CTRNN	63
3.7 Integrazione numerica: CTRNN come DTRNN .	64
3.8 Esempi di realizzazione in macchina	66
3.8.1 Dinamica di una semplice rete CTRNN . .	66
3.9 Ricerca dei parametri: Apprendimento	74
3.9.1 Algoritmi genetici	75
3.9.2 Codifica e scelta degli operatori	80
3.10 Esempio di apprendimento di attrattori	82
4 Gerarchie di reti	86
4.1 Reti come controllo di un agente	86
4.2 Rete di livello inferiore	88

<i>INDICE</i>	4
4.2.1 Primitive motorie	88
4.2.2 Studio del comportamento dinamico	89
4.2.3 Codifica della rete	90
4.2.4 Simulazione	91
4.2.5 Valutazione della rete	93
4.2.6 Ambienti di addestramento	95
4.2.7 Rete Addestrata	96
4.2.8 Nodi Comuni	97
4.2.9 Estensione alla rete di livello inferiore	106
4.3 Rete di livello superiore	108
4.4 Conclusioni e possibili sviluppi	112
A Stabilità Numerica	115
A.1 Metodi di integrazione numerica	115
A.2 Stabilità e Stabilità assoluta numerica	117
A.3 Metodo Forward di Eulero	118
Bibliografia	121

Capitolo 1

Introduzione

1.1 Obiettivi

Il presente lavoro di tesi ha come obiettivo lo studio e la realizzazione di un modello di controllo senso-motorio biologicamente plausibile ovvero un sistema, biologicamente ispirato, che sia in grado di controllare un robot che interagisce con un ambiente. Tale sistema deve realizzare una “mappatura” dall’informazione sensoriale del robot in azioni nel mondo reale. In Intelligenza Artificiale ci sono diverse strategie per la costruzione di tali sistemi di controllo (vedi [JM05]):

1. strategie basate su una pianificazione (*planner-based*). Questa strategia implica che il sistema contiene un modello di mondo centralizzato che consenta la verifica dell’informazione sensoriale la generazione di azioni nell’ambiente.
2. strategie basate su un approccio totalmente reattivo (*purely reactive*). All’opposto delle strategie *planner-based*, in questo

caso una strategia di controllo è costituita da una collezione di coppie *condizione* - *azione*, quindi regole reattive senza stati interni.

3. strategie ibride. In questo caso il sistema è composto di due parti comunicanti ma sostanzialmente indipendenti, una *planner-based*, un'altra *purely reactive*. In genere i processi reattivi vengono impiegati per assicurare l'immediata sicurezza del robot, mentre i processi basati su piano selezionano sequenze di azioni da eseguire.

(a) anche se spesso trattate come classe a parte, all'interno delle strategie ibride si possono includere le strategie basate sul comportamento (*behaviour-based*). Tali strategie si basano su architetture parallele e distribuite che in qualche modo concorrono fra loro al comportamento di tutto il sistema.

Per dare un significato al concetto di *behaviour* in robotica si può partire dalla definizione di *agente* (vedi [RN95]):

Definizione 1.1 *Si definisce **agente** qualsiasi ente che può **percepire** il suo ambiente attraverso **sensori** e **agire** in tale ambiente per mezzo di **effettori**.*

Utilizzando la definizione di agente possiamo introdurre la nozione di *agente razionale*:

Definizione 1.2 Si definisce *agente razionale* un agente che compie l'azione più appropriata secondo un certo criterio di misura di prestazione (*performance measure*).

A questo punto si può introdurre il concetto di *behaviour*:

Definizione 1.3 Un *behaviour* è l'azione compiuta da un agente razionale data una sequenza di percetti.

L'approccio seguito nel lavoro di tesi si può far rientrare nella categoria delle strategie *behaviour-based*. Il punto di partenza del lavoro sono i dati biologici, in particolare quelli riguardanti le aree motorie del cervello (vedi [LR00, FGB⁺01, RL01, GFFR96, MFF⁺97, RA98]), e i corrispondenti tentativi di modellizzazione computazionale (vedi [OA03, FA98, PSM05]). Essi suggeriscono una divisione funzionale delle aree e la creazione di un modello parallelo e distribuito con parti connesse e concorrenti fra di loro, ciascuna espressione di *behaviour* e in qualche modo influenzabile dall'interazione con le altre. Si sceglie di focalizzare l'attenzione sul circuito AIP-F5, riccamente oggetto di studio in letteratura, ed in particolare sulla parte di esso che riguarda la codifica e l'esecuzione di un piano motorio. Si definisce una *architettura software* che realizzi in maniera astratta e del tutto generale un modello che tenga insieme le principali caratteristiche estrapolate dagli esperimenti biologici. Per la realizzazione *effettiva* di tale struttura, si è seguita una felice intuizione di Tani (vedi [PT04a, PT04b]), che riesce nella codifica di un *behaviour* attraverso un particolare

tipo di reti neurali, le CTRNN (Reti neurali a tempo continuo - *Continuous Time Recurrent Neural Networks*), strutturate in una rete di basso livello, che costituisce il vero e proprio controllo, influenzabile *in cascata* dal comportamento di un'altra rete ad essa connessa. Tutto ciò suggerisce la possibilità della creazione di una *gerarchia* di reti CTRNN interagenti fra loro attraverso la “messa in comune” di alcuni elementi, e tale spunto è il filo conduttore del lavoro di tesi. Per studiare proprietà e caratteristiche di tali reti, al fine di produrre un *cablaggio* opportuno, guidato oltre che da basi sperimentali anche da presupposti teorici, il passo successivo è stato analizzare i lavori di Beer (vedi [YB94, Bee95]) che inquadrano tali reti all'interno della teoria dei sistemi dinamici. Tale inquadramento teorico sarà utile alla progettazione di tali reti e allo studio successivo delle proprietà che queste reti esibiranno. Infine gli esperimenti eseguiti tramite un *simulatore software di ambienti* (vedi [KH04]) costituiranno una dimostrazione di come tali reti siano sufficientemente ricche per codificare un piano¹: tali esperimenti saranno eseguiti progettando reti adatte a controllare un robot (nel caso degli esperimenti un *Pioneer3DX*) sfruttando la sensoristica per il pilotaggio di tale robot in un ambiente.

¹Come sarà chiaro nel seguito della trattazione, qui intendiamo per piano una *sequenza di behaviour*.

1.2 Capitoli

I capitoli sono strutturati in modo da percorrere le varie tappe fino all'implementazione effettiva delle reti costituenti parte del modello.

- Nel secondo capitolo si esegue una rassegna dei principali modelli di controllo senso-motori presenti in letteratura che si ispirano alle aree motorie della corteccia cerebrale. Da essi si è cercato di astrarne le caratteristiche principali in modo da definire un'*architettura software* che le implementi. In particolare all'interno di questa architettura si focalizza l'attenzione su alcuni elementi più propriamente legati al controllo motorio. Lo scopo è quello di realizzare anche tali elementi in una maniera biologicamente plausibile e la scelta cade, prendendo spunto dai recenti lavori di Tani, su un particolare tipo di reti neurali, le CTRNN (*Continuous Time Recurrent Neural Networks*).
- Nel terzo capitolo verranno introdotte le reti CTRNN, inquadrare nella teoria dei sistemi dinamici, studiate le principali caratteristiche ed eseguiti esperimenti semplici su reti di piccolo cablaggio. In particolare per l'implementazione in macchina si farà uso di un metodo di integrazione numerica, il *metodo forward di Eulero*, che rende le reti implementate matematicamente identiche ad un altro tipo di reti neurali, le DTRNN (*Discrete Time Recurrent Neural*

Networks). Si analizzeranno, formalmente comportamenti dinamici di tali reti: tali comportamenti permetteranno di governare i robot oggetto degli esperimenti del quarto capitolo. Infine si descrive il metodo con cui sono stati tarati i parametri componenti le CTRNN.

- Nel quarto capitolo, si riproduce una opportuna rete CTRNN per il *governo* di un robot *Pioneer3DX* saggiata sia in un ambiente di simulazione software *open-source* (progetto *Player-Stage-Gazebo*) sia in un ambiente reale. I risultati sperimentali vanno nella direzione auspicata, ovvero si esibiscono reti opportunamente costruite, che interagiscono in cascata, e che risultano capaci di codificare piani in termini di sequenze di behaviour. Termineremo presentando le conclusioni del lavoro svolto e alcuni possibili sviluppi futuri.

Capitolo 2

Modello di sistema senso-motorio

2.1 Sistemi di controllo: ispirazione biologica

Nella comunità scientifica c'è un sempre più forte interesse per la progettazione e lo sviluppo di *sistemi di controllo senso-motorio ispirati al biologico*. Tale crescita di interesse è principalmente motivata dalla disponibilità di sempre più numerosi dati sperimentali concernenti il funzionamento delle aree del cervello dei primati. Essi sembrano portare ad una parcellizzazione del *sistema nervoso centrale*¹ in numerose aree *funzionalmente dedicate*. Di particolare interesse sono gli esperimenti eseguiti sulle *aree-motorie* della corteccia cerebrale, che hanno messo in evidenza come tali aree siano adibite non solo ad una semplice esecuzione dei movimenti, ma a funzioni che corrispondono ad attività ad un più alto “livello

¹Per sistema nervoso centrale (SNC) si intende l'*encefalo* ed il *midollo spinale*. L'encefalo è composto da *diencefalo*, *cervelletto*, *tronco encefalico* e i due *emisferi cerebrali*. I due emisferi sono rivestiti da uno strato di cellule spesso dai 2 ai 6 mm che prende il nome di *corteccia*.

cognitivo”. Nell’ambito del controllo senso-motorio, sembra essere sempre più evidente che quello che era considerato meramente un percorso corticale in cui l’informazione visiva serviva a codificare la zona spaziale in cui eseguire il movimento (percorso del *where* - vedi figura 2.1) ha cominciato ad essere considerato piuttosto il percorso che codifica come il movimento debba essere eseguito (percorso dell’*how*). Gli esperimenti a cui faremo riferimento sono eseguiti in maggior misura su alcune specie di primati che vanno sotto il nome di *macaco rhesus* seguendo un protocollo standard formalizzato per tali esperimenti, il *protocollo Sakata* (vedi figura 2.2). Essi registrano la frequenza con cui i neuroni di queste aree rispondono a stimoli a cui sono sottoposti: quindi quando parleremo di attività registrata in queste aree ci riferiremo proprio a attività registrata negli elementi principali costituenti la corteccia, ovvero i *neuroni*². La corteccia cerebrale risulta strutturata in 4 lobi: *frontale*, *parietale*, *temporale*, *occipitale* (vedi figura 2.3). La classificazione delle aree del cervello a cui si farà riferimento è quella in [MLR91].

2.1.1 Sistema motorio corticale

Focalizziamo l’attenzione su quelle aree del cervello dei primati che vengono classificate come facenti parte del *Sistema motorio corticale* (vedi [RLM98]). Due zone del cervello ne fanno essen-

²Stiamo facendo riferimento alla frequenza media di spike dei neuroni. Tale concetto sarà affrontato nel capitolo 3.

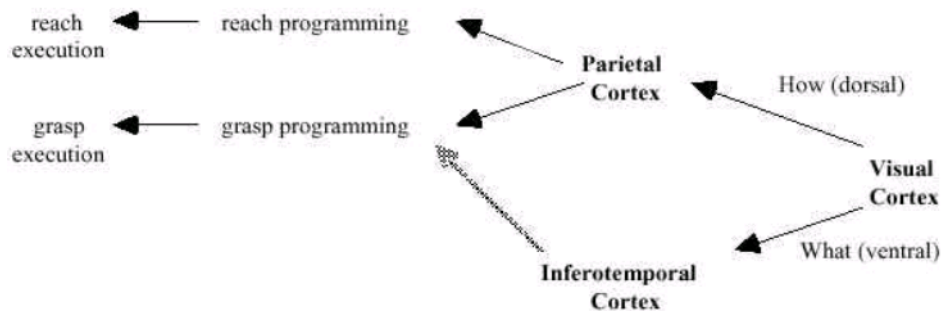


Figura 2.1: La dicotomia tra localizzazione (*where*) e identificazione (*what*) fu distinta da Ungerleider e Mishkin (vedi [UM82]) nella corteccia dei primati in due percorsi entrambi originantisi nell'area visiva primaria V1:

- percorso *ventrale* $V1 \rightarrow V2 \rightarrow V4 \rightarrow IT$
- percorso *dorsale* dall'area V1 all'area PP

Nel lavoro citato il percorso ventrale era quello ritenuto responsabile dell'identificazione dell'oggetto, quello dorsale della localizzazione. Successivi studi (vedi [GM92]) hanno suggerito in realtà che quest'ultimo debba essere più propriamente ritenuto responsabile della codifica di *come* l'azione debba avvenire per andare a buon fine (percorso dell'*how*).

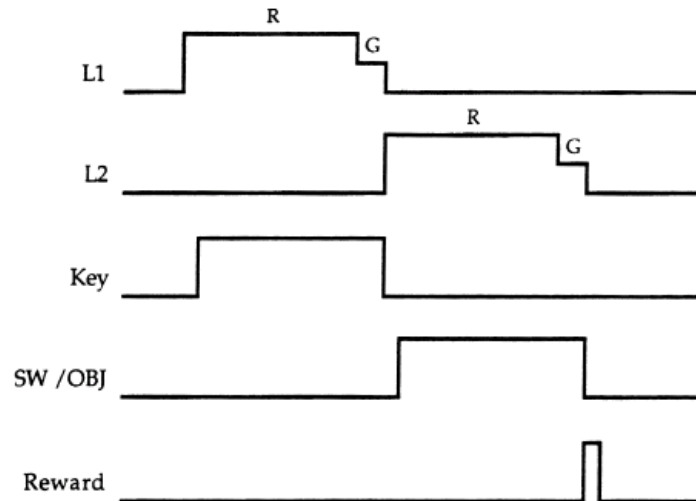


Figura 2.2: Il *Protocollo Sakata* così come è descritto in [TMGMA90]: tale protocollo è stato formalizzato per studiare proprietà dei neuroni relative ad *azioni di presa* (*grasping*). All'accensione del led L1 la scimmia risponde posizionando la sua mano sul pannello (*touch pad*) posto di fronte ad essa: questa azione pone inizio alla “fase di contatto” (*key*). Quando il led L1 cambia colore la scimmia è addestrata per iniziare l'azione di presa. Un secondo led L2 si accende nel momento in cui la scimmia non è più in contatto col pannello e comincia la “fase oggetto” (*object phase*) in cui la scimmia compie l'azione di presa e manipola l'oggetto. Quando il led L2 cambia colore la scimmia è addestrata a rilasciare l'oggetto e ad allontanare la mano. Se il compito è eseguito correttamente il macaco viene premiato con del cibo (*reward*).

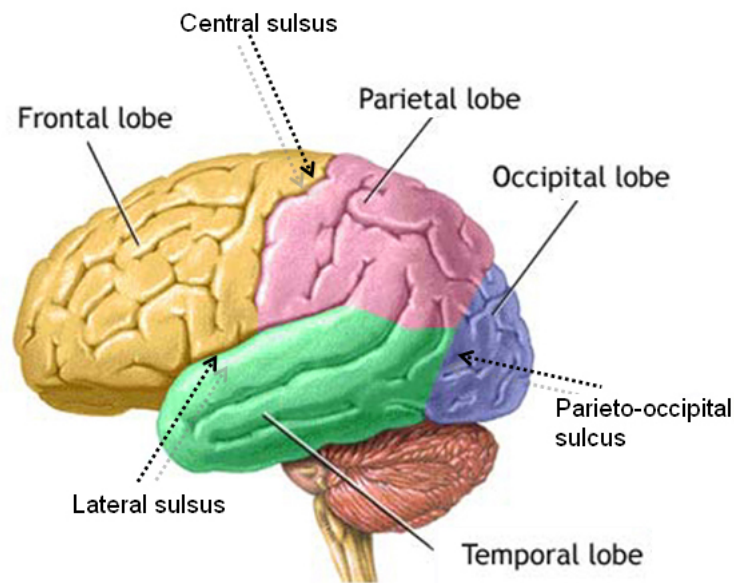


Figura 2.3: Divisione architettónica in *lobi* del cervello ottenuta attraverso scanalature di separazione dette solchi (*sulcus*). La parte anteriore al *solco centrale* è detta *lobo frontale*. La parte al di sotto del *solco laterale* fino al *solco parieto-occipitale* è detta *lobo temporale*. La zona compresa tra solco centrale, solco laterale e solco parieto-occipitale è detta *lobo parietale*. Infine la zona situata posteriormente al solco parieto-occipitale è detta *lobo occipitale*.

zialmente parte: la *corteccia agranulare frontale*³, e la *corteccia parietale posteriore* (vedi figura 2.4).

2.1.2 Corteccia Agranulare Frontale

La corteccia agranulare frontale che prende anche il nome di *corteccia motoria* secondo la divisione di Broadmann⁴ (vedi [Kor09]) era composta di due aree: *Area 4* e *Area 6*. Nella nuova ripartizione a cui si fa riferimento, l'Area 4 corrisponde all'area F1 (la F sta per *frontale*) - ovvero la *corteccia motoria primaria* - mentre esperimenti mirati hanno consentito una suddivisione dell'Area 6 in ulteriori 6 aree, sempre numerate: F2, F3, F4, F5 ed F6. Le prime 5 (F1, F2, F3, F4 ed F5) sono connesse con zone dell'area parietale e si osserva che tipicamente ciascuna area parietale è connessa in modo più marcato (ma non in maniera esclusiva) con un'area motoria: in prima approssimazione, quindi, ogni area parietale può

³La corteccia cerebrale è la parte superiore dei due emisferi cerebrali. In genere è caratterizzata dall'essere composta da 6 strati (layers) a seconda del tipo di composizione del tessuto:

1. strato *molecolare*
2. strato *granulare esterno*
3. strato *piramidale esterno*
4. strato *granulare interno*
5. strato *piramidale interno*
6. strato *multiforme*

La corteccia si dice *agranulare* se manca del quarto strato, ovvero lo strato granulare interno.

⁴A Broadmann si deve una classificazione delle aree del cervello sostanzialmente ispirata dalla composizione dei tessuti nervosi.

essere considerata connessa solo con un'unica area motoria. Facciamo una breve panoramica di quel che è noto di queste aree della corteccia motoria:

2.1.2.1 Corteccia premotoria mesiale

Il termine mesiale fa riferimento alla posizione che occupa: infatti è la parte dell'area 6 che si trova tra (*in mezzo a*) i due emisferi. È divisa nelle aree F3 ed F6⁵.

- L'**area F3** è supposta contenere una rappresentazione completa del movimento del corpo. I movimenti coinvolti con la sua attivazione sono soprattutto quelli prossimali ed assiali. I neuroni F3 mostrano frequentemente risposte *somatosensoriali*⁶. Ha connessioni con le aree motorie F1, F2, F4 ed F5. Si suppone che il ruolo dell'area F3 sia cruciale nel controllo della postura, e in particolare in movimenti di aggiustamento che precedano movimenti volontari.
- L'**area F6** risponde soprattutto a stimoli visivi. Inoltre mostra una forte attività sia durante la preparazione dell'azione sia quando oggetti passibili di un movimento di presa entrano nel campo d'azione dell'animale ad una distanza che consenta l'azione. Non è connessa con l'area F1, ma con tutte le altre aree premotorie. L'area F6 è supposta essere

⁵A queste aree si fa anche riferimento rispettivamente come SMA (*Supplementar Motor Area*) e pre-SMA (*pre-Supplementar Motor Area*).

⁶Stimoli attinenti al corpo.

responsabile del controllo dell'azione di moto nel senso della decisione del *quando* e *se* eseguire il movimento selezionato.

2.1.2.2 Corteccia premotoria dorsale

La parola dorsale si riferisce alla posizione di tale area situata sulla zona superiore dell'emisfero cerebrale. È composta dalle aree F2 ed F7

- L'**area F2** è collegata con l'area F1. Recenti osservazioni hanno messo in evidenza come ci siano due zone differenti all'interno dell'area con connessioni parietali differenti:
 - una zona posta dorsalmente e ventralmente la cavità (dimple) superiore precentrale, che prende appunto il nome di F2 dimple. I neuroni dell'area si attivano per stimoli *propriocettivi*⁷ e sembra essere collegata alla rappresentazione della gamba e del braccio. Tale zona è collegata alle aree parietali PEc e PEip, aree coinvolte in elaborazione somatosensoriale di alto livello;
 - l'altra zona, sostanzialmente la parte rostrale e ventrale dell'F2 che prende il nome appunto di F2vr, è connessa con l'area parietale VIP.
- L'**area F7** è connessa con le area F2 ed F6. In essa è presente una zona detta SEF - *Supplementary Eye Field* - che

⁷Per *proprioccezione* si intende il senso della posizione del proprio corpo.

contribuisce al controllo dei movimenti degli occhi, in particolare delle *saccadi*⁸. Tale area potrebbe essere coinvolta nella codifica delle posizioni dell'oggetto nello spazio e per la coordinazione dei movimenti braccio-gamba.

2.1.2.3 Corteccia premotoria ventrale

La parola ventrale si riferisce alla posizione dell'area situata nella zona dell'emisfero cerebrale rivolta verso l'esterno. È composta dalle aree F4 ed F5

- l'**area F4** contiene una rappresentazione dei movimenti della faccia, del collo e del braccio. Una parte di essi rispondono a stimoli somatosensoriali, un'altra parte sono *bimodali*, ovvero rispondono a stimoli sia visuali sia somatosensoriali. Ha forti connessioni con un'area della corteccia posteriore, l'area VIP (*Ventral Intra Parietal area*).
- Recenti scoperte citoarchitettoniche hanno evidenziato come l'**area F5** sia in realtà divisa in due settori, una nella parte posteriore rialzata del solco arcuato inferiore (F5 *bank*), l'altra posizionata sulla convessità adiacente il solco arcuato (F5 *convexity*). L'attività dei neuroni F5 bank mostra che essi sono fortemente attivi durante precise azioni "finalizzate ad un goal"⁹ (*goal-directed*) e in particolare azioni di pre-

⁸Le saccadi sono movimenti veloci e simultanei di entrambi gli occhi nella stessa direzione.

⁹La parola *goal* (obiettivo) si riferisce ad un'informazione che un agente razionale può

sa (*grasping*), tenuta (*holding*), strappo (*tearing*) e manipolazione. Alcuni di essi rispondono pure alla presentazione di oggetti tridimensionali (*fixation*) e discernono attraverso forma, dimensione ed orientazione dell'oggetto in questione: tutto questo anche non in presenza di un successivo movimento di presa. Questa è l'attività tipica di quella classe di neuroni F5 detti *canonici*. L'interpretazione che viene data (vedi [RL01]) a questo comportamento è che tali neuroni codifichino *azioni potenziali* che il sistema può compiere in funzione dell'oggetto presentato. La presenza di tali azioni potenziali suggerisce pure la presenza di un *sistema di controllo* in grado di fornire un segnale di *start* per provocarne la reale esecuzione. Per movimenti che riguardano il braccio il sistema che invia questo segnale sembra essere costituito dalla parte di corteccia motoria che prende il nome di F6. I neuroni dell'F5 convexity risultano indistinguibili dai neuroni F5 bank, tranne in due caratteristiche peculiari:

- non sono attivi alla presentazione dell'oggetto, ovvero durante la fissazione;
- sono attivi non solo quando la scimmia compie l'azione, ma anche quando essa osserva la stessa azione eseguita da altri individui. Tale proprietà viene detta specchio

possedere, oltre ai percetti, da utilizzare al fine di compiere l'azione giusta: ad esempio scegliere ad un bivio se andare a destra o a sinistra a seconda della destinazione da raggiungere. Con azioni finalizzate ad un goal qui si fa riferimento ad azioni come prendere un pezzo di cibo, spezzare un pezzo di carta, mantenere un oggetto.

(*mirror*) e dà il nome a questi neuroni che la posseggono (neuroni mirror).

La proprietà mirror suggerisce l'ipotesi che il sistema motorio funga pure da *interprete* delle azioni da compiere: ovvero che a partire dai dati dell'oggetto riesca a formulare un programma valido di presa a prescindere da chi sia il soggetto dell'azione. Tale proprietà, in letteratura, è molto spesso correlata quindi alla capacità di *imitazione* e/o *riconoscimento d'azione*. Questa scoperta ha suscitato vasto interesse nella comunità scientifica. Gli F5 convexity sono fortemente connessi all'area PF.

2.1.3 Corteccia parietale posteriore

La zona della corteccia parietale posteriore è divisa in un *lobo parietale inferiore* (IPL) ed in un *lobo parietale superiore* (SPL). Accenniamo alle proprietà di alcune aree connesse alla corteccia motoria.

2.1.3.1 Lobo parietale inferiore

Tale zona è suddivisa funzionalmente in diverse aree (vedi figura 2.4) di cui citeremo le più rilevanti al fine della trattazione:

- **L'area MIP** (*Mesial Intra-Parietal area*) è situata nella parte caudale della cavità mediale del solco intraparietale. In

quest'area ci sono neuroni con proprietà visuali, altri con proprietà bimodali. Tale area è fortemente connessa con l'area F2_{vr}.

- **L'area VIP** (*Ventral Intra-Parietal area*) occupa la parte inferiore (*fundus*) del solco intraparietale. Pure i suoi neuroni sono divisi in totalmente visuali e bimodali. Come già detto quest'area è fortemente connessa con l'area F4.
- **L'Area PF** (*Parieto-Frontal area*) è situata sulla convessità della lobo parietale inferiore. Pochi studi sono stati effettuati su quest'area. È fortemente connessa con l'area F5 convexity e come per i neuroni dell'F5, pure parte dei neuroni dell'area PF sembrano avere la proprietà mirror. Tale area, inoltre, sembra esser parte di una sorta di *memoria di lavoro* (*working memory*) dell'interazione oggetto - comando di presa appropriato.
- **L'area AIP** (*Anterior Intra-Parietal Area*) occupa la parte rostrale del rilievo laterale del solco intraparietale ed è connessa fortemente con l'area F5 *bank*. In tale area sembrano esserci neuroni codificanti oggetti utili alle azioni di presa. La loro attività non è influenzata dalla posizione dell'oggetto nello spazio.

I neuroni dell'area AIP sono stati classificati in tre diverse categorie:

- neuroni attivi prevalentemente in fase di movimento (*motor-dominant*);
- neuroni attivi prevalentemente in fase di visione (*visual-dominant*);
- neuroni attivi sia in fase di movimento, sia in fase di visione (*visuo-motor*).

I neuroni *motor-dominant*, sparano durante le azioni di presa (*grasp*) e tenuta (*holding*) dell'oggetto sia alla luce sia al buio ma non sparano durante la fase della fissazione (*fixation*) dell'oggetto. Neuroni *visual-dominant* sparano durante azioni di presa alla luce e durante la fissazione dell'oggetto ma non durante movimenti al buio. Neuroni *visuo-motor* sparano sia per movimenti sia alla luce sia al buio ma con una maggiore significatività per movimenti alla luce. Gli esperimenti hanno suggerito l'ipotesi che una parte della popolazione dei neuroni dell'AIP *codifichi* le proprietà visive di oggetti tridimensionali.

2.2 Circuiti Parieto-premotori

Abbiamo già accennato che ciascuna area parietale ha connessioni quasi esclusive con una singola area premotoria parieto-dipendente (da F1 ad F5). Sicché coppie di aree premotorie-parietali formano una serie di *circuiti anatomici indipendenti* l'uno dall'altro (vedi

Tabella 2.1: Coppie di circuiti premotorio-parietali nella corteccia della scimmia macaco. In corsivo sono segnate le aree visuali, in grassetto le aree bimodali.

Aree motorie	Connessioni Predominanti
F1	PE
F2 dimple	PEc, PEip
F2 _{vr}	<i>MIP</i>
F3	PEci, 24d
F4	<i>VIP</i>
F5	<i>AIP</i> , <i>PF</i>

tabella 2.1). È stato ipotizzato che ciascuno di questi circuiti sia responsabile di una specifica trasformazione senso-motoria. Un'azione di presa comporta la necessità di molteplici trasformazioni senso-motorie per essere eseguita ed una tale organizzazione anatomica suggerisce che *il sistema motorio corticale sia in grado di elaborare questi processi in maniera parallela.*

[...] *each parieto-premotor circuit is dedicated to specific sensorimotor transformation [...] The visuomotor transformations for action may imply simultaneous control of different visual mechanisms [...]* *The anatomical organization of the parietofrontal connections suggest that the cortical motor system is able to process in parallel all these aspects related to the control of arm and hand movements.* [ML01]

Siamo ancora lontani da una perfetta comprensione delle funzionalità di tali circuiti. Ci sono indizi sperimentali che ci consentono di identificare alcune caratteristiche di essi:

- Il circuito VIP-F4 codifica lo spazio *peripersonale* usando un sistema di coordinate *egocentrico*, ovvero che ha come centro di riferimento parti del corpo del soggetto che compie l'azione. In tal modo trasforma l'informazione sulla posizione dell'oggetto in opportuni movimenti verso di esso.
- Il circuito MIP-F2vr potrebbe avere una funzione di *controllo della fase di trasporto dalla mano verso l'oggetto*, controllo effettuato sulla base di informazione somatosensoriale e visuale.

Un discorso a parte va fatto per il circuito AIP-F5, o più correttamente, per i circuiti AIP-F5bank e PF-F5 convexity. I dati sperimentali disponibili sono in misura molto maggiore e su di essi si è scritto molto dopo l'affascinante scoperta di neuroni con proprietà mirror.

2.3 Circuito AIP-F5

Dopo una panoramica sull'intero sistema motorio corticale, restringiamo l'attenzione, dunque, sul circuito AIP-F5, che gli esperimenti hanno mostrato di essere fortemente coinvolto nelle *azioni di presa (grasping)*, a partire dall'informazione visiva, fino alla pianificazione del movimento e all'esecuzione. L'interesse in questo

circuito è che sembra essere la parte cruciale per la pianificazione delle azioni; è stato suggerito che sottopopolazioni dell'area F5 vadano a codificare primitive motorie, e l'intera area F5 sia in realtà un “vocabolario” in cui attingere per la formazione di piani completi di presa¹⁰:

F5 contains a storage (“vocabulary”) of motor actions [...] This Motor vocabulary is constituted of “words”, each of which is represented by a population of F5 neurons [...] These words select specific “motor prototypes” such as, for example the configuration of fingers necessary for the precision grip [...] Thus, each action is represented by specific populations of neurons that code the action at different degrees of abstraction. [RL01]

[...] each time an object is observed, its visual features are automatically (no matter whether a movement is intended or not) “translated” into a “vocabulary” of motor acts stored in area F5. [GMLK00]

In letteratura sono presenti vari tentativi di modellizzazione computazionale del sistema motorio corticale, tra i quali menzioniamo il modello FARS - *Fagg Arbib Rizzolatti Sakata* (vedi [FA98]), il

¹⁰Si possono distinguere diversi tipi di presa tra cui esempi significativi sono le *prese di precisione*, le *prese di forza* e le *prese laterali*. Per ognuna di queste prese un piano riguarda svariate variabili come la preparazione alla postura della mano (*preshape*), la velocità di apertura delle dita, la velocità di approccio all'oggetto, e così via. I dati sperimentali suggeriscono la presenza di sottoaree dell'area F5 codificanti ognuna di queste prese.

modello MSN1 - *Mirror Neuron System* 1 (vedi [OA03]) ed il modello MEP - *Mirror Expected Perception* (vedi [PSM05]). Tali modelli computazionali hanno tentato di cogliere le relazioni funzionali tra le aree, al fine di dare un possibile quadro di come l'informazione sensoriale venga codificata. È accordo comune che il primo passo del processo che va dalla rappresentazione dell'oggetto all'esecuzione del movimento, avvenga nei neuroni dell'AIP *visual-dominant*: essi sono ritenuti responsabili delle estrazioni delle caratteristiche¹¹ dell'oggetto tridimensionale, informazione che verrà trasmessa all'area F5.

- Il modello FARS suggerisce che le informazioni su multiple *affordances* (vedi nota 11) dell'oggetto vengano passate all'area F5 appunto per attivare i corrispondenti possibile *prototipi motori* che codificano vari tipi di presa. I neuroni F5 canonici esercitano una qualche *attività di rinforzo* per l'attività dei neuroni *visuo-motor* dell'area AIP: l'interpretazione di tale attività, mantenuta durante il movimento, è che tali neuroni fungano come una sorta di *memoria attiva* (dinamica) durante l'intera esecuzione dell'azione.
- La scelta del tipo di presa dipende oltre che dalle proprietà dell'oggetto anche dal *significato* dell'oggetto stesso. Un

¹¹Tali caratteristiche estratte prendono il nome di *affordances* termine mutuato da J. J. Gibson (vedi [Gib66, GG55]) per indicare informazione visiva di basso livello, in opposizione ad informazioni di alto livello che hanno a che vedere con *identificazione* e *significato* degli oggetti codificati dal processo visivo. Quando si fa uso di tale termine, è implicito allora che si ha a che fare con informazioni geometriche di basso livello ma che sono sufficienti ad impostare un piano motorio corretto.

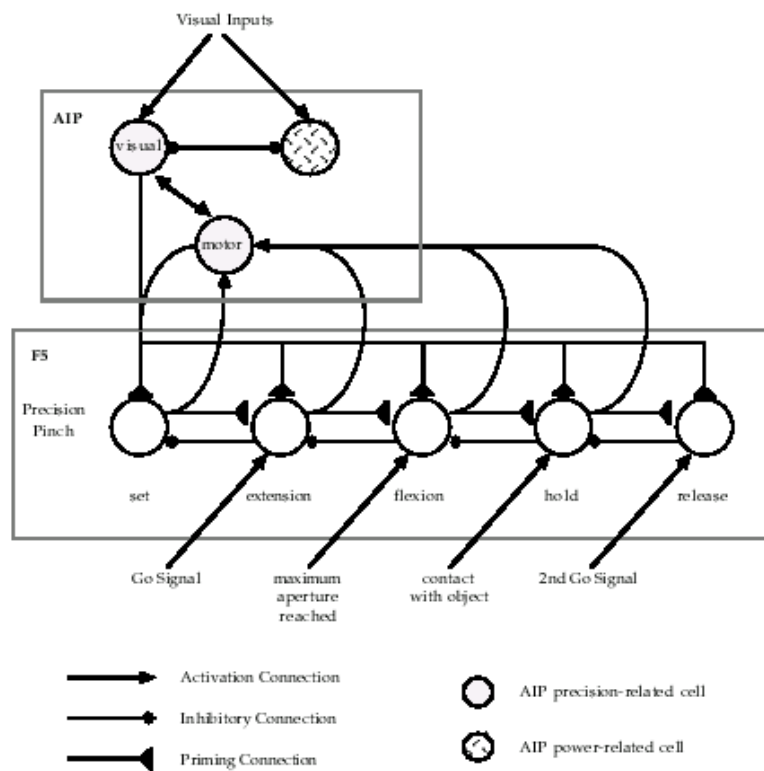


Figura 2.5: Dal *modello FARS* (vedi [FA98]), interazione tra l'area AIP e l'area F5. Sottopopolazioni di F5 codificano parte del piano di presa e vengono attivate in sequenza da sinistra a destra.

esempio lampante è l'azione di prendere una tazza: se la si vuole spostare la presa migliore è effettuata dal corpo o dal bordo superiore; se il fine è bere, la presa corretta va effettuata dal manico. Quindi al fine di una presa corretta, è necessaria informazione proveniente da circuiti codificanti il significato degli oggetti (percorso *ventrale* del flusso visivo detto del *what*), circuiti che si trovano nel lobo inferotemporale (IT). Rizzolatti (vedi [RL01]) suggerisce che dato che anatomicamente esistono connessioni dirette dall'IT all'AIP, sia proprio quest'ultima ad avere la “parola finale” sulla selezione del tipo di presa.

- Nel modello MSN1 (vedi [OA03]), ideale continuazione del modello FARS, si fa l'ipotesi che la proprietà mirror sia evoluta nell'area F5 per fornire un sistema di *feedback* per consentire il controllo sulla mano (*hand state*) durante le azioni di presa.
- Nel modello MEP si fa l'ipotesi che i neuroni specchio siano parte di un meccanismo di anticipazione (*percezione attesa*, vedi [DTL⁺03b, DTL⁺03a]): si può immaginare che tali neuroni sparino in corrispondenza di un *matching* tra il *percepto atteso* costituito dalla successiva azione *in fieri*, dalla sequenza di azioni codificate che costituiscono un piano, e la *percezione reale* di cui potrebbe essere l'area PF a tenere traccia, fortemente connessa con l'area F5 convexity, e facente

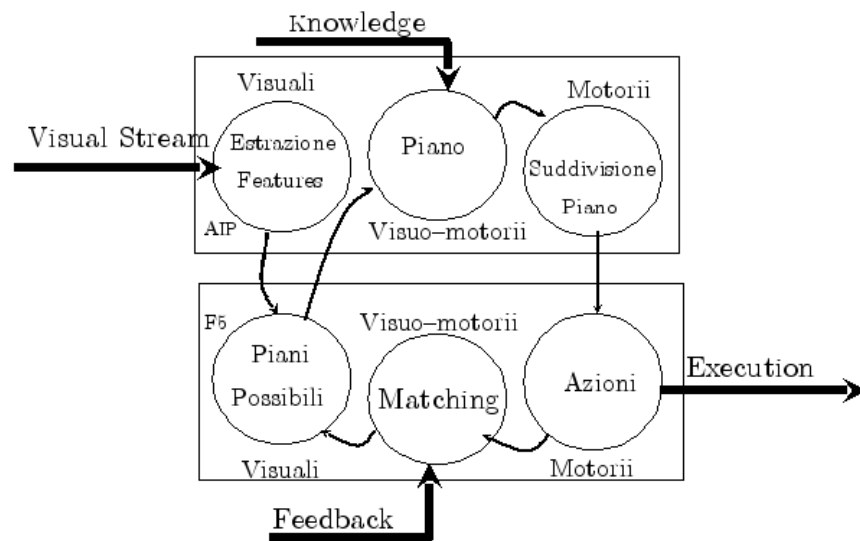


Figura 2.6: Schema di caratteristiche estrapolate dal circuito AIP-F5

parte, come si è visto, di una sorta di memoria di lavoro. Tale meccanismo di matching è interpretato come base per la classificazione e il riconoscimento di azioni.

2.3.1 Modello a blocchi

Alla luce di quanto detto nei paragrafi precedenti, dunque, sembra plausibile che un generico modello motorio ispirato alle caratteristiche biologiche del sistema motorio corticale e in particolare al circuito AIP,PF-F5, potrebbe essere costituito dai moduli o blocchi esposti in seguito. La descrizione delle possibili operazioni e componenti sarà formalizzata mediante un *linguaggio di programmazione ad oggetti*, Java[Mic] con l'utilizzo di *Interfacce*, *Oggetti astratti* e *metodi virtuali*. I moduli componenti un generico

modello motorio potrebbero essere:

- un modulo per l'estrazione delle caratteristiche (*Extractor* - vedi tabella 2.2)

Tabella 2.2: Interfaccia Extractor in Java

```
public interface Extractor {  
    public void retrieveFeatures(Sensor sensor);  
}
```

a partire da dati sensorii (*Sensor* - vedi tabella 2.3)

Tabella 2.3: Interfaccia Sensor in Java

```
public interface Sensor {  
    public Object retrieveData();  
}
```

- un modulo che individui i possibili piani a partire dalla caratteristiche estratte (*Selector* - vedi 2.4)

Tabella 2.4: Interfaccia Selector in Java

```
public interface Extractor {  
    public void getPlans(Extractor extractor);  
}
```

- un modulo che selezioni il piano tra i possibili in base ad ulte-

riore informazione se necessario (*Knower*), e che lo conservi in memoria (*Planner* - vedi 2.5)

Tabella 2.5: Interfaccia Planner in Java

```
public interface Planner {
    public void choosePlan(Selector selector, Knower knower);
}
```

- un modulo che suddivida il piano in sotto-piani o sotto-sequenze da eseguire (*Engine* - vedi 2.6)

Tabella 2.6: Interfaccia Engine in Java

```
public interface Engine {
    public MotorPlan getMove(Planner planner);
}
```

- un modulo che mandi in esecuzione (*Effector* - vedi 2.7)

Tabella 2.7: Interfaccia Effector in Java

```
public interface Effector {
    public boolean executeData(MotorPlan motor_plan);
}
```

l'azione corrente (*MotorPlan*) (vedi tabella 2.8)

Tabella 2.8: Interfaccia MotorPlan in Java

```

public interface MotorPlan {
    public Object getCommands();

    public void SetProperties(Object data);

    void updatePlan (Object new_info);
}

```

Tali moduli costituiscono una *libreria di oggetti astratti* in Java, e possono essere visti come gli elementi costituenti di un generico modello di controllo senso-motorio ispirato al biologico. Ad esempio, a partire da tale libreria si può immaginare di implementare blocchi interagenti come quelli in figura 2.6, che approssimano un'interazione parieto-prefrontale (vedi tabelle 2.9 e 2.10). Con questi blocchi è possibile implementare un sistema di controllo ispirato al MEP *algorithm* (vedi 2.11), tratto dal modello MEP appunto, con un oggetto astratto *MotorSystem*.

Nel presente lavoro si è focalizzata l'attenzione su uno dei moduli di base che riguarda la *codifica* e l'*esecuzione di un piano* (MotorPlan). Gli sforzi si sono rivolti, dunque, nel cercare un modo biologicamente plausibile per implementare tale codifica.

Tabella 2.9: Oggetto astratto con funzionalità modellate sull'area F5

```
public abstract class PremotorArea implements Selector
    Engine,
    Matcher {
    public MotorPlan nextMove(Planner planner,
        Reader reader,
        Effector effector,
        Timing timing) {
        MotorPlan action = this.getMove(planner);
        if (timing.isReady() && this.match(this,reader))
            effector.executeData(action);
        return action;
    }

    public abstract boolean noMoreMoves();
}
```

Tabella 2.10: Oggetto astratto con funzionalità modellate sull'area AIP

```
public abstract class ParietalArea implements Extractor
    Planner {
    public void planify (Selector selector,
        Knower knower) {
        selector.getPlans(this);
        this.choosePlan(selector,knower);
    }
}
```

Tabella 2.11: Oggetto Java che implementa una versione dell'algoritmo MEP (vedi [PSM05]).

```

public abstract class public classMotor_System {
    public void motor_action (Sensor area_V,
                              Knower it) {
        Reader pf,
        Timing f6,
        Effector f1 {
        do {
            AIP.retrieveFeatures(area_V);
            AIP.planify(F5,it);
            do {
                F5.nextMove(AIP, pf, f1, f6);
            }
            while (! F5.noMoreMoves());
        }
        while (! this.actionExecuted());
    }

    public abstract boolean actionExecuted();
}

```

2.4 Codifica del piano: sequenza di primitive

Nel presente lavoro di tesi, come punto di partenza per la codifica di un piano motorio si è scelto di utilizzare un particolare tipo di reti neurali, le reti CTRNN - *Continuous Time Recurrent Neural Networks* (vedi capitolo 3). Tale scelta nasce seguendo i lavori di Tani (vedi [PT04a, PT04b]) che utilizzano tali reti.

2.4.1 Reti di Tani

A partire dai recenti lavori di Tani, nel seguito si è analizzata la possibilità di codificare un comportamento (*behaviour*), per mezzo di un particolare tipo di reti neurali, le CTRNN. Tali reti neurali, come vedremo nel prossimo capitolo, risultano essere una approssimazione *biologicamente ispirata* di un circuito nervoso. Da questi lavori risulta che è possibile controllare una specifica rete CTRNN, mettendo in comune parte del circuito che lo compone, con un'altra rete dello stesso tipo. L'analogia con i circuiti biologici cerebrali è evidente. Possiamo dunque identificare due livelli di analogie delle CTRNN con un sistema biologico a due diversi livelli di aggregazione:

- *analogia di composizione*, per come sono strutturate: le parti elementari che la compongono sono ispirate al neurone biologico;
- *analogia di funzionamento*, perché tra reti costruite in modo da realizzare funzionalità analoghe alle aree cerebrali si

possono organizzare opportuni cablaggi in modo che il comportamento sia un'approssimazione di quello biologico.

2.4.2 Gerarchia di Reti

Dunque l'idea di reti neurali interconnesse che si influenzano fra loro rispecchia quello che suggerisce ciò che si è compreso dell'architettura cerebrale. L'insieme di tali reti andrebbe a produrre un sistema parallelo e distribuito, in cui ogni parte ha un ruolo funzionale influenzato o mutato attraverso la connessione di nodi provenienti da altre reti. Tale approccio può essere fatto rientrare in robotica all'interno delle strategie *behaviour-based* (vedi paragrafo 1.1). La concatenazione di tali reti permetterebbe la creazione di sistemi complessi in grado di riprodurre comportamento "intelligente". Tale sistema di concatenazione può essere sfruttato per realizzare *effettivamente* le parti del modello riguardanti la codifica del piano. L'idea è quella di codificare in una rete di *basso livello* le *primitive motorie* che compongono le azioni possibili di un agente dotato di possibilità di movimento. Tali primitive dunque sono quelle che in robotica prendono il nome di *behaviour*, ed è possibile immaginare un modello di reti *gerarchicamente connesse* che riesca a codificare un piano. Il punto di partenza è la nostra definizione di lavoro di *piano motorio*:

Definizione 2.1 Per *piano motorio* p intendiamo una sequen-

za definita di behaviour

$$p \equiv \{b_1, \dots, b_i, \dots\}$$

Data una rete R_1 codificante un insieme di K_1 *behaviour*

$$B = \{b_i\}_1^{K_1}$$

facciamo l'ipotesi che sia possibile costruire una rete R_2 di più alto livello, opportunamente connessa alla rete R_1 , tale da influenzare il comportamento di quest'ultima a compiere una determinata sequenza definita di behaviour

$$b_1, b_2, \dots, b_k \dots$$

Il comportamento della rete R_2 provocherà per il sistema, dunque, l'esecuzione di un determinato piano p . Il variare del comportamento della rete R_2 comporta variazioni nella sequenza di behaviour, e quindi un *cambiamento del piano* in un nuovo piano p' . Quindi il risultato è che *la rete R_2 è in grado di codificare un insieme di K_2 piani*:

$$S = \{p_i\}_1^{K_2}$$

Ci si può spingere oltre ed immaginare una nuova rete R_3 ancora opportunamente cablata con la rete R_2 capace di influenzare tale rete per poterne produrre un cambiamento tale da farla passare dalla codifica di un piano ad un altro. In tal modo la rete R_3 risulterebbe codificare una *libreria di possibili piani* (per inciso

quello che sembra fare l'area F5). Iterando tale ragionamento è evidente che l'opportuna connessione in cascata di queste reti (o addirittura in *loop*) potrebbe permettere la codifica di *capacità* via via di livello superiore. Nel lavoro di tesi eseguiremo degli esperimenti confortanti in tale direzione riportati nel capitolo 4. Prima di affrontarli il prossimo capitolo sarà interamente incentrato a guardare più da vicino le reti CTRNN che compongono i *mattoni* dell'intero modello.

Capitolo 3

Reti Neurali a tempo continuo

3.1 Neurone biologico

Il *Sistema Nervoso* è un apparato dotato di elevata capacità operativa, costituito essenzialmente di un elevato numero di parti elementari, funzionante come un tutto unico; ma è anche un insieme di sistemi e di livelli funzionali gerarchicamente disposti e deputati a ben determinate attività secondo una precisa localizzazione spaziale.

Il *neurone* o *cellula nervosa* è l'unità anatomica e funzionale del sistema nervoso. La sua principale caratteristica è la capacità di trasformare, a determinate condizioni, stati di eccitamento in segnali (*spari*) che vengono trasmessi ad altri neuroni o ad organi efferenti in una complicata serie di interconnessioni. La sua struttura può essere schematizzata sinteticamente in un corpo principale (*soma*), al cui interno è presente *liquido intracellulare*, e che è avvolto da una *membrana*, e da cui si dipartono più ramificazioni d'ingresso (*dendriti*) ed una ramificazione d'uscita (*assone*) re-

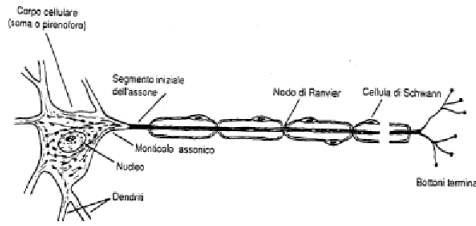


Figura 3.1: Struttura di un neurone

sponsabile della propagazione del segnale verso altri neuroni; l'assone infatti termina in ramificazioni dette *sinapsi*, ognuna delle quali a sua volta termina in un *bottone sinaptico*, dotato di *vescicole*, attraverso le quali passano i *neurotrasmettitori* responsabili della trasmissione del segnale. Per convenzione si parla di celle *pre-sinaptiche* per indicare i neuroni che inviano segnale e *celle post-sinaptiche* per indicare neuroni che lo ricevono. Un singolo neurone nella corteccia dei vertebrati può connettersi anche a 10^4 celle post-sinaptiche.

I meccanismi di trasmissione del segnale sono essenzialmente chimici. All'interno della membrana, e nel liquido esterno ad essa (*liquido extracellulare*), sono presenti ioni Na^+ , K^+ e Cl^- , distribuiti in modo da avere una differenza di potenziale negativa (più precisamente, il polo negativo è interno al neurone, quello positivo esterno ad esso) tipicamente uguale a $\Delta V_r \approx -70mV$ detta *potenziale a riposo*. La permeabilità della membrana è regolata dalle *pompe sodio-potassio* al suo interno che tendono a ristabilire la d.d.p. a riposo.

Durante il riposo la membrana è fondamentalmente permeabile

soltanto agli ioni K^+ . La migrazione di ioni positivi all'interno del liquido intracellulare, in risposta agli stimoli ricevuti dai neurotrasmettitori, fa aumentare il potenziale post-sinaptico dovuto alla somma di tutti i contributi pre-sinaptici in entrata, fino ad una soglia $\Delta V_S \approx -50mV$, raggiunta la quale viene prodotto quello che va sotto il nome di *potenziale d'azione* (A.P.) o *spike* tramite un meccanismo di ritorno allo stato di riposo che porta alla produzione dello sparo, e alla conseguente propagazione dell'impulso.

Questo meccanismo a soglia è definito “tutto o niente” nel senso che non è incrementale ma avviene se e solo se si raggiunge il valore soglia del potenziale. In questa fase di attività si ha un notevole aumento nella membrana della permeabilità al sodio che porta ad una rapidissimo aumento del potenziale. La durata di quest'aumento di permeabilità detta *tempo refrattario* è di circa $1 \div 2ms$; refrattario perché durante tale intervallo il neurone non risponde ad altri eventuali stimoli esterni¹. Di conseguenza la *frequenza massima di spike* è di circa $500 Hz$ ². Trascorso questo tempo la permeabilità della membrana al sodio ritorna pressoché a zero mentre quella al potassio aumenta consentendone la migrazione all'esterno fino a riportare il potenziale nella condizione iniziale di riposo con una breve escursione al di sotto di esso. Il picco

¹Si osserva che il periodo di *assoluta refrattarietà* è seguito da un periodo di *relativa refrattarietà* in cui è difficile, ma non impossibile, che avvenga un nuovo sparo.

²Il valore riferito è misurato per neuroni di tipo *senso-motorio*. Per aree corticali superiori il tempo refrattario aumenta e le frequenze di sparo registrate sono al massimo dell'ordine dei $150 Hz$; addirittura in alcune zone non sono superiori ai 30 o $40 Hz$.

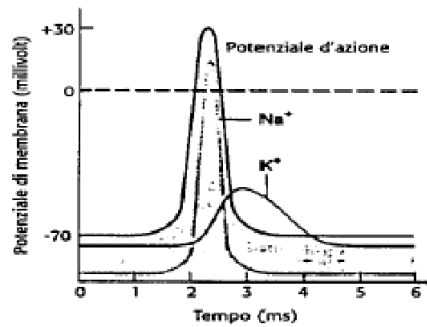


Figura 3.2: *Potenziale d'azione*. La figura mostra come lo sparo sia essenzialmente dovuto all'aumento della permeabilità al sodio da parte della membrana, mentre il ritorno allo stato di riposo venga favorito da un aumento della permeabilità al potassio.

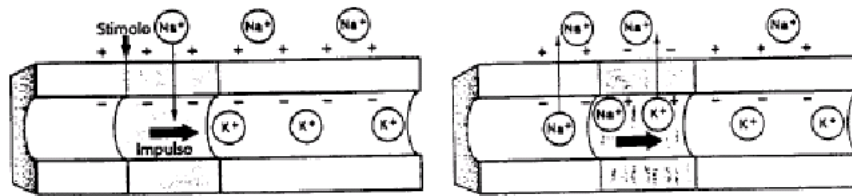


Figura 3.3: *Propagazione del segnale*. Una volta prodotto lo stimolo il segnale si propaga lungo l'assone attraverso due tratti contigui.

che raggiunge la d.d.p. è di circa 35 mV ; lo stimolo si trasmette lungo tratti contigui dell'assone. Sperimentalmente si verifica che lo spostamento lungo l'assone è direttamente proporzionale al diametro dell'assone stesso che a sua volta è proporzionale alla sua lunghezza; quindi la velocità di propagazione del segnale può essere considerata almeno in prima approssimazione costante con un valore di circa $\approx 120\text{ m/s}$.

3.1.1 Dinamica dei neuroni e sinapsi

Una descrizione, in prima approssimazione, della dinamica di un neurone può essere data dai seguenti passi:

1. Il segnale, risultato del meccanismo “tutto o niente” precedentemente descritto, si propaga lungo l’assone. L’ampiezza dell’impulso viaggiante è stabile ed è replicato a ogni ramificazione dell’assone. L’ampiezza è dell’ordine delle decine di millivolts. Con il neurone a riposo non si rinviene alcun segnale nell’assone, piuttosto sussiste un potenziale detto appunto di riposo. La presenza di un impulso nell’assone blocca la possibilità della trasmissione di un secondo impulso.
2. Il segnale giunge nelle terminazioni finali dell’assone, le sinapsi, causando la secrezione di neuro-trasmettitori da parte delle vescicole site nei bottoni sinaptici.
3. I neuro-trasmettitori, oltre le sinapsi, si legano ai recettori dei dendriti del neurone post-sinaptico, causando una corrente ionica nella membrana. L’intensità di corrente indotta per l’attivazione del neurone pre-sinaptico è un parametro che specifica l’efficacia della sinapsi.
4. Ogni singolo contributo, all’incirca un millivolt, partecipa alla formazione del potenziale post-sinaptico. Tali contributi possono essere *eccitatori* – se fanno aumentare il potenziale di membrana incrementando la possibilità che il neurone ven-

ga attivato – o *inibitori*, e in tal caso riducono la possibilità di una attivazione.

5. Se la somma totale dei potenziali post-sinaptici pervenuti in un intervallo di tempo è maggiore di una certa soglia, si ha il *potenziale d'azione*.

3.2 Modello fisico di neurone

Possiamo approssimare il comportamento biologico dal punto di vista della teoria dei *circuiti elettrici*. La membrana cellulare di un neurone può essere schematizzata in prima approssimazione come un circuito RC parallelo (vedi figura 3.4), (modello di Hodgkin - Huxley, [HH52])

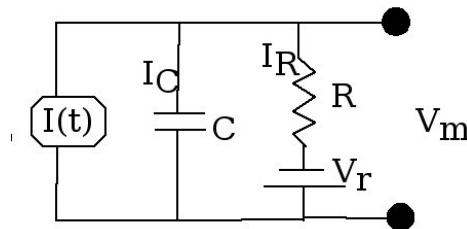


Figura 3.4: Schematizzazione della membrana cellulare come circuito RC parallelo

in cui gli elementi in gioco sono:

- il potenziale di membrana V_M
- la resistenza di membrana R_M

- un potenziale di riposo V_R
- una capacità di membrana C_M
- una corrente esterna che rappresenta il contributo di corrente stimolata dagli altri neuroni che denotiamo con $I(t)$.

Applicando la prima legge di Kirchoff

$$I(t) = I_C + I_R$$

con $I_C = C_M \frac{dV_M}{dt}$ e $I_R = \frac{V_M(t) - V_R}{R_M}$ da cui otteniamo l'equazione

$$R_M C_M \frac{dV_M(t)}{dt} = -V_M(t) + R_M I(t) + V_R \quad (3.1)$$

In cui $R_M C_M = \tau$ è la costante di tempo secondo la quale varia il potenziale di membrana. La stessa equazione possiamo riscriverla quindi come:

$$\frac{dV_M(t)}{dt} = \frac{1}{\tau} (-V_M(t) + R_M I(t) + V_R) \quad (3.2)$$

Si può approssimare il contributo di corrente proveniente dall'apertura delle vescicole e la conseguente immissione dei neurotrasmettitori come l'immissione nel circuito di una corrente a gradino che sia nulla prima del tempo iniziale e un valore costante I_0 dopo:

$$I(t) = \begin{cases} 0 & t \leq 0 \\ I_0 & t > 0 \end{cases}$$

possiamo risolvere il circuito per $t > 0$. La soluzione dell'omogena

associata $R_M C_M \frac{dV_M(t)}{dt} + V_M(t) = 0$ è

$$V_M(t) = k e^{-\frac{t}{RC}}$$

mentre una soluzione particolare è

$$V_M(t) = V_R + R_M I_0$$

quindi una soluzione dell'equazione 3.1 è

$$V_M(t) = k e^{-\frac{t}{RC}} + V_R + R_M I_0$$

Imponendo la condizione

$$V_M(0) = V_R$$

otteniamo

$$V_R = k e^{-\frac{t}{RC}} + V_R + R_M I_0 \implies k = -R_M I_0 e^{-\frac{t}{RC}}$$

da cui possiamo riscrivere la soluzione

$$V_M(t) = -R_M I_0 e^{-\frac{t}{RC}} + V_R + R_M I_0 \quad (3.3)$$

Dalla soluzione dell'equazione risulta chiaro che *lo stimolo prodotto dall'attività dei neuroni afferenti non fa variare istantaneamente l'ampiezza della differenza di potenziale $V_M(t)$ ai capi della membrana ma si manifesta con un certo ritardo dipendente dalla costante di tempo τ .*

3.3 Modello di rete: CTRNN

Una rete neurale a tempo continuo (CTRNN - *Continuous Time Recurrent Neural Network*) è un particolare tipo di rete neurale ricorrente. Essa costituisce un sistema composto da N parti elementari q_i con $i \in 1, \dots, N$ dette *nodi* (o *neuroni*), tutte *connesse*³ tra di loro e ad S elementi esterni detti *Input*. Matematicamente parlando una rete R del tipo CTRNN è definita da una *quadrupla*:

$$R \equiv \{Q, I, W, W'\}$$

In cui

$$Q \equiv \{q_i\}_{i=1}^N$$

è l'insieme dei nodi costituenti la rete. L'applicazione W associa a ciascuna connessione tra gli elementi della rete un numero reale, detti *peso*, e definisce la relazione:

$$W : (q_i, q_j) \in Q \times Q \rightarrow w_{ij} \in \mathbb{R}$$

per un totale di $N \times N$ pesi. L'insieme I è l'insieme degli S *input*,

$$I \equiv \{I_k\}_{k=1}^S$$

³Una *connessione* c_{ij} dall'elemento x_j all'elemento q_i è una coppia ordinata $c_{ij} \equiv (q_i, x_j)$ con $q_i \in Q$ e $x_j \in Q \cup I$. L'insieme di tutte le connessioni

$$C \equiv (Q \cup I) \times Q$$

contiene $N \times N + S \times N$ coppie ordinate.

con $I_k \in \mathbb{R}$ valori su cui la rete è *forzata* esternamente, i quali a loro volta sono connessi ai nodi tramite $N \times S$ pesi definiti similmente a quanto scritto precedentemente dalla relazione W' :

$$W' (q_i, I_k) \in Q \times I \rightarrow w'_{ik} \in \mathbb{R}$$

3.3.1 Legge di aggiornamento

La caratteristica fondamentale di una CTRNN è che la dinamica dei neuroni segue una legge di aggiornamento descritta da una *equazione differenziale* del tipo

$$\frac{dy_i}{dt} = f_i (y_1, \dots, y_N) \equiv \frac{1}{\tau_i} \left(-y_i + \sum_{j=1}^N w_{ij} \sigma (y_j - \theta_j) + \sum_{k=1}^S w'_{ik} I_k \right) \quad (3.4)$$

con i che varia da 1 ad N . Tutte le variabili in gioco appartengono ai Reali. Rispetto all' i -esimo neurone:

- y_i è lo stato (detto *attivazione*) del neurone,
- τ_i una *costante di tempo*,
- w_{ij} il *peso* associato alla connessione verso l' i -esimo neurone proveniente dal j -esimo neurone,
- I_k il k -esimo *input esterno* pesato tramite il valore w_{ik} ,

- σ è la *funzione logistica* di attivazione⁴, e $\sigma(y_j - \theta_j)$ sarà detto *valore d'uscita* del neurone j
- θ_i è un valore di calibrazione detto *soglia* o *bias*.

Scritta in forma vettoriale la medesima equazione diviene:

$$\boldsymbol{\tau} \frac{d\mathbf{y}}{dt} = -\mathbf{y} + \mathbf{W}\boldsymbol{\sigma}(\mathbf{y} + \boldsymbol{\theta}) + \mathbf{I}$$

in cui $\boldsymbol{\tau}$, $d\mathbf{y}/dt$, $\mathbf{y}$, $\boldsymbol{\theta}$ e $\mathbf{I}$, sono vettori N dimensionali, mentre $\mathbf{W}$ è una matrice $N \times N$.

Confrontando l'equazione 3.2 con la 3.4 si possono individuare analogie che potrebbero essere sfruttate per dare un'interpretazione biologica agli elementi matematici costituenti la legge di aggiornamento di una CTRNN. Fatta l'ipotesi che il termine $R_M I(t)$ nell'equazione 3.2 che modella il potenziale di membrana rappresenti la *differenza di potenziale "indotta" dalle frequenze di spike dei neuroni afferenti* possiamo interpretare i termini dell'equazione 3.4 nel modo seguente:

- l'attivazione y_i come il *potenziale di membrana* del neurone i ;

⁴Una forma generale per la classe delle funzioni di attivazione è

$$\sigma_{\alpha,\beta,\mu}(x) = \frac{\alpha}{1 + e^{-\mu x}} + \beta$$

con $\alpha, \mu \in \mathbb{R}^+$, $\beta \in \mathbb{R}$. Per tutto il lavoro, comunque, useremo la funzione logistica standard, detta *sigmoide*:

$$\sigma_{1,0,1}(x) = \frac{1}{1 + e^{-x}}$$

- i pesi w_{ij} come il *massimo numero di vescicole che si possono aprire* su un bottone sinaptico del neurone j afferente al neurone i all'arrivo di un potenziale d'azione;
- l'uscita $\sigma(y_j - \theta_j)$ come la *d.d.p. per vescicola aperta* “in-dotta” dalla frequenza di spike del *neurone j*

Differentemente dalle reti feed-forward che sono puramente reattive, e pertanto definite *statiche*, queste reti sono dette *dinamiche*, perché capaci di essere influenzate dalla storia di un processo in esecuzione non solamente dagli input (stimoli) del momento.

3.4 Rete CTRNN come Sistema di Controllo

Nella comunità scientifica (vedi [Bee95, YB94, BF02, FM94]) c'è un interesse crescente per l'utilizzo di reti neurali dinamiche per il controllo di *robot* attraverso l'*evoluzione temporale* di tali reti. Partiamo dalla definizione di *stato* della rete.

Definizione 3.1 *Definiamo lo stato $S(t) \in [0, 1]^N$ di una rete al tempo t , come l' N -pla dei valori dell'attività σ_i al tempo t dei singoli neuroni i .*

$$S(t) \equiv \left\{ \sigma_1(t), \sigma_2(t), \dots, \sigma_N(t) \right\}$$

A partire dallo stato, definiamo cosa intendiamo per *traiettoria* di una rete.

Definizione 3.2 *Si definisce traiettoria di una CTRNN l'insieme dei punti in un iperpiano R^N individuati al variare di t , dagli stati $S(t)$.*

Abbiamo accennato che data una rete essa può essere utilizzata come controllo per un robot. Diamo un'idea di come questo possa avvenire: in base agli input della rete (che possono provenire dai sensori del robot), che supponiamo vengano passati all'istante t_0 viene definita una traiettoria $S(t)$ per i punti $t > t_0$. Ogni stato $S(\bar{t})$ codificato in modo opportuno viene passato a degli effettori del robot che interagirà con l'ambiente esterno in relazione all'evoluzione della traiettoria. Risulta evidente, dunque, che l'oggetto di studio debba essere la traiettoria individuata dalla rete per poter ottenere il controllo voluto sul robot. Proprio per studiare le evoluzioni temporali delle traiettorie individuate da una rete CTRNN, il prossimo passo sarà inquadrare tali reti nella *teoria dei sistemi dinamici* in modo da avere un supporto teorico per la progettazione, costruzione e la valutazione di reti che individuino traiettorie opportune per lo scopo di controllare il movimento di robot nella maniera "intelligente" desiderata.

3.5 Rete CTRNN come Sistema Dinamico

Si può studiare una rete CTRNN dal punto di vista della teoria dei sistemi dinamici: in questo modo possiamo sfruttare teoremi e definizioni del tutto generali riguardanti questa teoria (vedi

[GH86]). Per far ciò osserviamo che l'equazione 3.4 costituisce dal punto di vista matematico un sistema di *equazioni differenziali ordinarie* che a loro volta possono essere guardate come un particolare sistema dinamico. I passaggi teorici sono chiariti nelle definizioni successive.

3.5.1 CTRNN come sistema di ODE

L'equazione 3.4 rappresenta un caso particolare di un sistema di *equazioni differenziali ordinarie* (*Ordinary Differential Equations*) del primo ordine. Nella sua forma più generica è un sistema di equazioni contenente derivate in un'unica variabile e si può esprimere con una formula del tipo:

$$\frac{d\mathbf{y}}{dt} = f(\mathbf{y}(t), t) \quad (3.5)$$

con $\mathbf{y}(t) \in \mathbb{C}^N$, $N \geq 1$ (anche se di solito si prende $\mathbf{y}(t) \in \mathbb{R}^N$). In questo modo si determina un sistema di equazioni differenziali ordinarie di *dimensione* N .

Quali sono le condizioni per l'*esistenza* e l'*unicità* delle soluzioni? Bisogna specificare le *condizioni iniziali* per il sistema 3.5 ovvero la descrizione iniziale per la soluzione $\mathbf{y}(t)$ al tempo $t = t_0$:

$$\mathbf{y}_0 \in \mathbb{C}^N \quad \text{al tempo } t = t_0 \quad \text{con } t \in [t_0, T]$$

per cui il sistema diviene:

$$\begin{cases} \mathbf{y}(0) = \mathbf{y}_0 \\ \frac{d\mathbf{y}}{dt} = f(\mathbf{y}(t), t) \end{cases} \quad (3.6)$$

Le condizioni (*di Cauchy*) di esistenza ed unicità della $\mathbf{y}(t)$ sono:

- f continua rispetto a t e ad $\mathbf{y}$
- f soddisfa la *condizione di Lipschitz*⁵ rispetto a $\mathbf{y}$

3.5.2 Sistema di ODE come sistema Dinamico

Un *sistema dinamico* S consiste di una tripla (T, S, ϕ^t) in cui T è l'insieme ordinato delle *coordinate temporali*, S è uno *spazio topologico*⁶ detto *spazio degli stati*, e ϕ_t l'*operatore evoluzione* con

$$\phi : (x, t) \in S \times T \rightarrow \phi(x, t) \in S$$

con $t \in T$ e $x \in S$. L'insieme T delle coordinate temporali può essere continuo ($T \subseteq \mathbb{R}$) o discreto ($T \subseteq \mathbb{C}$), finito o infinito dimensionale. L'operatore evoluzione ϕ^t può essere deterministico o stocastico.

⁵Una funzione f soddisfa la condizione di Lipschitz se:

$\exists L \geq 0$ tale che $\forall \mathbf{y}, \mathbf{z} \in \mathbb{C}^N$ e $t \in [0, T] \Rightarrow \|f(\mathbf{y}, t) - f(\mathbf{z}, t)\| \leq L \|\mathbf{y} - \mathbf{z}\|$

⁶Uno spazio topologico X è un insieme su cui è definita una *topologia* T .

Una topologia T su X è un *insieme di sottoinsiemi di X* (che prendono il nome di *aperti*) con le seguenti proprietà:

1. L'insieme vuoto ϕ e X appartengono ad T
2. $\forall U_i \in T$ si ha $\bigcup_{i=1}^K U_i \in T$ con K arbitrario
3. $\forall U_i \in T$ si ha $\bigcap_{i=1}^K U_i \in T$ con K finito

Un sistema di ODE può essere visto come un sistema dinamico in cui lo spazio degli stati S è l'insieme dei punti $\mathbf{y}$ per cui è definita la f dell'equazione 3.5. L'insieme T è l'insieme delle coordinate temporali, mentre l'operatore evoluzione ϕ è definito (in maniera univoca per le condizioni di Cauchy 3.6) dalle soluzioni dell'equazione differenziale.

Fissato il punto iniziale x_0 , $\phi(x_0, t) = \phi^t(x_0)$ rappresenta una soluzione dell'equazione differenziale. Al variare delle x in S abbiamo un fascio di soluzioni dell'equazione. A questo punto siamo in grado di introdurre le definizioni di *stabilità*, *punti fissi*, e *attrattori*.

3.5.3 Stabilità

Definizione 3.3 Una **soluzione** $\phi^t(x_0)$ di un sistema dinamico si dice **stabile** se qualsiasi soluzione prossima ad essa al tempo $t = 0$, lo rimane anche in seguito:

- formulazione per intorni⁷:

$$\forall U_1 (\phi^t(x_0)) \exists U_2 (\phi^0(x_0) = x_0) : \phi^t(U_2) \subset U_1 \forall t \quad (3.7)$$

Definizione 3.4 Una soluzione $\phi^t(x_0)$ si dice **asintoticamente stabile** se è stabile ed inoltre se ogni soluzione vicina si può scegliere in maniera tale che per $t \rightarrow \infty$ essa coincida con la

⁷Indichiamo con $U(x)$ un intorno del punto x

soluzione data; l'ulteriore condizione da dare è: alla stabilitàva aggiunta un'ulteriore condizione

- $\forall U_1 (\phi^t(x_0)) \exists U_2 (\phi^0(x_0)) : \phi(U_2) \subset U_1 \forall t$ (stabilità)
-

$$\bigcap_{t \geq 0} \phi^t(U_2) = \phi^t(x_0)$$

3.5.4 Punti fissi

Definizione 3.5 Si definisce **punto fisso** (o di **equilibrio**, **singolare**, **stazionario**) di un sistema dinamico (S, μ, ϕ) , un punto x^* per il quale vale

$$\phi(x^*) = x^*$$

Definizione 3.6 Un punto fisso x^* è **stabile** se:

$$\forall U_1(x^*) \quad \exists U_2(x^*) \subset U_1 : \phi(U_2) \in U_1$$

Un punto fisso x^* è **asintoticamente stabile** se è stabile e vale inoltre che

$$\bigcap_{t \geq 0} \phi^t(U_2) = x^*$$

Utile alla trattazione sarà pure la nozione di *superficie di stabilità*:

Definizione 3.7 Si definisce *superficie di stabilità* di un sistema dinamico il luogo dei punti del sistema che verificano la condizione di equilibrio stabile.

3.5.5 Comportamenti al limite e regolarità

Di seguito introduciamo la nozione di attrattore premessa da vari tipi di comportamenti regolari e al limite. Si parte dalla definizione di un punto periodico per cui esiste

Definizione 3.8 *Un punto $\bar{x}$ di S si dice **periodico** se*

$$\exists T > 0 : \phi^T(\bar{x}) = \bar{x}$$

Definizione 3.9 *Un insieme $U \subset S$ si dice **invariante (in avanti)** se vale*

$$\phi(U) \subset U$$

Un concetto ancora più generale degli insieme invarianti in avanti sono i seguenti

Definizione 3.10 *Un insieme W si dice **Non-Wandering** se vale,*

$$\exists t \quad \phi^t(W) \cap W \neq \emptyset$$

Definizione 3.11 *Si dice **insieme limite** del sistema dinamico, l'insieme I_L di tutti i punti dello spazio S dati per un'evoluzione per $t \rightarrow \infty$:*

$$\forall x \in S \quad \bigcap_{t \geq 0} \phi^t(x) = I_L$$

Si osservi che l'insieme limite è invariante: $\phi(I_L) \subset I_L$

Definizione 3.12 *Un insieme chiuso A invariante in avanti è un **insieme attrattivo** per il sistema dinamico (S, μ, ϕ) , se esiste un aperto $U \supset A$ tale che:*

- $\phi(U) \subset U$ – ovvero U è invariante in avanti
- $\bigcap_{t \geq 0} \phi^t(U) = A$

Definizione 3.13 *Un insieme chiuso invariante T è detto **topologicamente transitivo** se*

$$\forall U, V \subset T \exists t : \phi^t(U) \cap V \neq \emptyset$$

Definizione 3.14 *Un **attrattore** è un insieme attrattivo che sia topologicamente transitivo*

Corollario 3.1 *Punti isolati non-wandering asintoticamente stabili sono attrattori.*

Corollario 3.2 *Punti di equilibrio asintoticamente stabili sono attrattori.*

Definizione 3.15 *Il **bacino di attrazione** B dell'attrattore A è definito come:*

$$B \equiv S \cap \left(\bigcup_{t \geq 0} \phi^{-t}(U) \right)$$

3.5.6 Biforcazioni

La teoria delle biforcazioni è una vasta area della matematica, ed una complessa formalizzazione di essa è al di fuori dagli scopi del lavoro. Essa studia il comportamento di un sistema dinamico al variare dei parametri del sistema. Se per piccoli cambiamenti intorno ad un parametro si ha un *cambiamento qualitativo* del sistema dinamico (ad es. cambiamento del numero di attrattori) allora in quel punto il sistema possiede una biforcazione. Per meglio chiarire il significato di questo concetto possiamo dare alcuni esempi di punti di biforcazione:

1. Biforcazione “PitchFork” (vedi figura 3.5). Dato il sistema dinamico ad un parametro μ

$$\dot{x} = \mu x - x^3$$

si ha per:

- $\mu < 0$ - un punto fisso stabile
- $\mu = 0$ - il punto fisso stabile diviene instabile, e ci sono due ulteriori punti fissi stabili

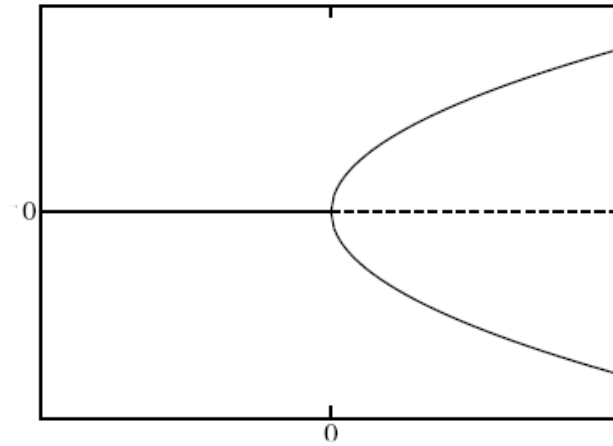


Figura 3.5: Diagramma di una biforcazione pitchfork. In ascisse c'è il parametro μ , in ordinata i valori assunti dalla $x(t)$ per $t \rightarrow +\infty$. Le soluzioni stabili sono in linea continua, quelle instabili sono tratteggiate.

1. Biforcazione “transcritica” (vedi figura 3.6). Dato il sistema

$$\dot{x} = \mu x - x^2$$

- $\mu \neq 0$ - due punti fissi stabili
- $\mu = 0$ - i due punti si toccano e cambiano stabilità

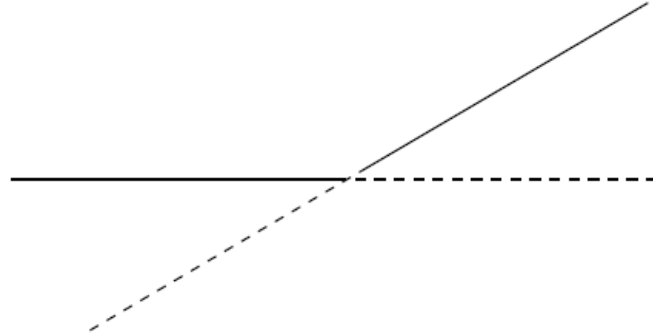


Figura 3.6: Diagramma di una biforcazione transcritica. I due punti si incontrano e cambiano di stabilità. Le linee tratteggiate indicano equilibrio instabile, quelle continue equilibrio stabile.

1. Biforcazione “saddle” (vedi figura 3.7). Dato il sistema

$$\dot{x} = \mu + x^2$$

- $\mu \leq 0$ - due punti fissi
- $\mu = 0$ - i due punti collidono e spariscono

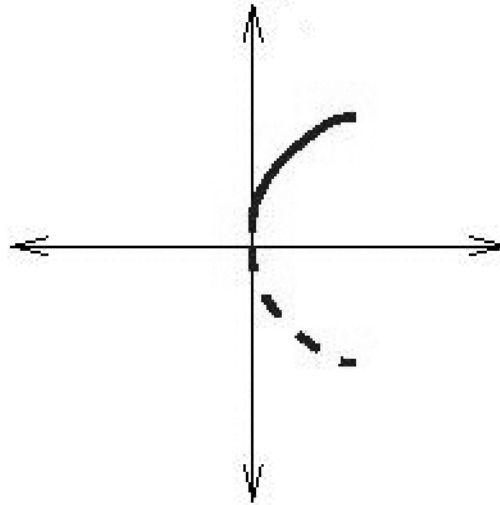


Figura 3.7: Diagramma di una biforcazione a sella. A sinistra non ci sono punti di equilibrio: i due punti di equilibrio stabile ed instabile a destra collidono e spariscono.

3.6 Capacità di memoria di una CTRNN

Nella costruzione di opportune reti CTRNN, si è tenuto conto del fondamentale risultato ottenuto da Beer [Bee95]

Teorema 3.1 *Il massimo numero di **punti di equilibrio** che può avere una rete CTRNN con N -neuroni è 3^N .*

In questo modo nella ricerca di punti di attrazione, dobbiamo costruire una rete con un numero sufficiente di nodi tale che possano esistere il numero di punti di equilibrio *asintoticamente stabili* sufficienti (non ci bastano punti di equilibrio qualsiasi). Naturalmente più ci sono più è probabile che ci siano conformazioni dei parametri tali da soddisfare le esigenze richieste. L'approccio

sarà quello di costruire reti di dimensioni *ragionevoli* e ricercare nello spazio dei parametri attraverso il metodo degli algoritmi genetici in modo da esibire un esemplare che dimostri l'esistenza della rete con le proprietà dinamiche volute.

3.7 Integrazione numerica: CTRNN come DTRNN

Per implementare in macchina il tipo di rete CTRNN occorre un modo per integrare numericamente le equazioni differenziali che compongono la legge di aggiornamento. Un metodo usato in letteratura (vedi [YB94, BF02]), è il *metodo Forward di Eulero* (vedi A) secondo il quale una funzione

$$\frac{dy}{dt} = f(y, t)$$

può essere approssimata con la formula:

$$y^{n+1} = y^n + \Delta t f(y^n, t_n)$$

scegliendo opportuni Δt per i quali la formula risulti assolutamente stabile. In questo modo in base alla formula A.5 applicata alla 3.4 otteniamo l'equazione discreta:

$$y_i^{n+1} = y_i^n + \frac{\Delta t}{\tau_i} \left(-y_i^n + \sum_{j=1}^N w_{ij} \sigma_j(y^n - \theta_j) + \sum_{k=1}^S w_{ik} I_k \right) \quad (3.8)$$

in cui n rappresenta il passo di calcolo e Δt la dimensione del passo.

Tale equazione coincide con la formulazione di una rete DTRNN (*Discrete Time Recurrent Neural Network*), ovvero una rete neurale in cui la legge di aggiornamento non è data da un'equazione differenziale. Infatti questa formula può essere inquadrata come una formula a sé stante, in cui abbiamo gli stessi elementi presenti nel tipo CTRNN, con l'unica differenza di considerare la costante di tempo come $\tau_i/\Delta t$. La rete implementata in macchina è dunque in realtà una rete DTRNN, che per un opportuno passo di integrazione Δt è una buona approssimazione di una rete CTRNN. Per tutti gli esperimenti realizzati è stata usata la condizione derivata da [HC95]

$$\Delta t \leq 2 \min\{\tau_i\} \text{ con } i \in 1, \dots, N \quad (3.9)$$

L'*evoluzione temporale* della rete avviene attraverso step discreti aggiornati in maniera *sincrona*. Ad ogni step n la rete si trova in uno stato ben definito $S(n)$. Per n sufficientemente grandi possiamo parlare di comportamenti asintotici della rete.

Riassumendo quindi, il programma in macchina che si è andato a realizzare può avere due chiavi di lettura:

1. Implementazione di una rete DTRNN
2. Approssimazione di una rete CTRNN, se viene soddisfatta la 3.9.

Nel seguito quindi ove si farà riferimento a reti DTRNN, saranno usati passi di integrazione opportuni in modo da poterle comunque considerare approssimazioni di CTRNN e poter parlare ove occorra di sistemi dinamici corrispondenti.

3.8 Esempi di realizzazione in macchina

Di seguito riportiamo alcune implementazioni di macchina che illustrano le potenzialità dell'architettura software che realizza una rete CTRNN.

3.8.1 Dinamica di una semplice rete CTRNN

Si è realizzata una *classe Java* che implementasse la formula 3.8. Si può studiare su piccole reti, come in [\[Bee95\]](#), il comportamento dinamico di esse. Partiamo dall'equazione 3.4 scritta per un solo neurone con un autoconnessione e con la costante $\tau = 1$

$$\frac{dy}{dt} = f(y) = -y + w\sigma(y - \theta) + I \quad (3.10)$$

La stabilità di quest'equazione è data per $f'(y) \leq 0$ ⁸ quindi

$$-1 + w\sigma'(y) \leq 0$$

Dato che $0 \leq \sigma'(y) \leq 1/4$, nel punto $w = 4$ risulta esserci un drastico cambiamento qualitativo del comportamento del sistema (biforcazione):

- caso $w < 4$: in questo caso vale sempre $f'(y) < 0$ e dunque le soluzioni dell'equazione sono valide per ogni y e ad ogni y corrisponde un'unica soluzione. Quali che siano le condizioni iniziali, fissati I , θ , e w il valore al limite dell'attività del neurone sarà un unico punto fisso. Non abbiamo una soluzione analitica della 3.10, ma possiamo ottenere, attraverso la sua approssimazione numerica e il programma java che la implementa, la forma della *superficie di stabilità*. Dato un valore

⁸Questa affermazione è facile da verificare. Infatti dato un sistema

$$\frac{dy}{dt} = f(y)$$

osserviamo che

$$\begin{aligned} \frac{dy}{dt} > 0 &\implies f(y) > 0 \\ \frac{dy}{dt} < 0 &\implies f(y) < 0 \end{aligned}$$

Dato un punto fisso y_0 (per cui vale $f(y_0) = 0$), la definizione 3.4 di asintotica stabilità implica che si tratti di un punto di minimo, ovvero debba accadere

$$\begin{cases} f(y) > 0 & \text{per } y > y_0 \\ f(y) < 0 & \text{per } y < y_0 \end{cases}$$

il che implica evidentemente che debba verificarsi

$$f'(y_0) < 0$$

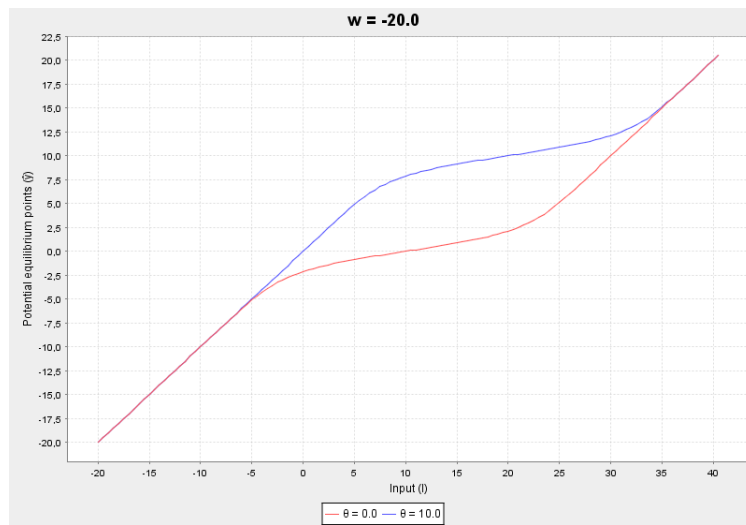


Figura 3.8: Superfici di stabilità per $w = -20$. Sono mostrate due superfici, in rosso quella per $\theta = 0$ e in blu quella per $\theta = 10$. Sono graficati i valori dell'attivazione y del neurone in corrispondenza di un valore di Input I che il neurone assume dopo un tempo sufficientemente lungo

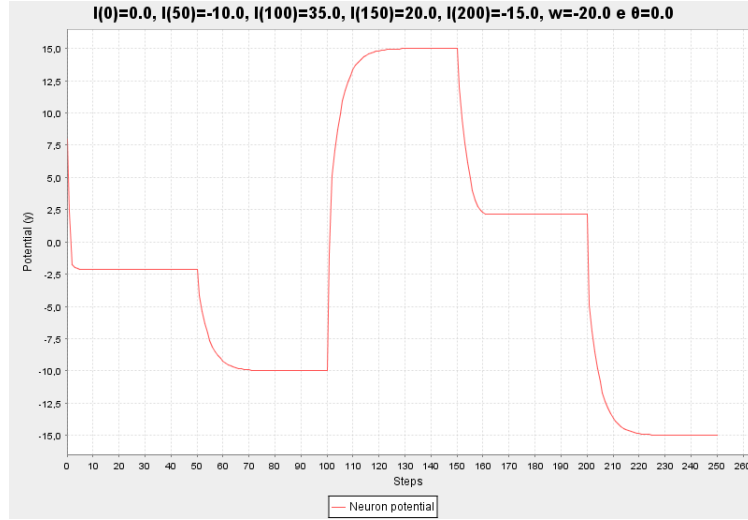


Figura 3.9: Comportamento dell'attivazione di un neurone $w = -20$ e $\theta = 0$. In questo caso è mostrata la variazione nel tempo dell'attivazione. Ogni 50 step viene variato il valore di Input I . Il valore dell'attivazione tende all'attrattore (punti appartenenti alla superficie di stabilità) dopo pochi step.

di Input I al neurone, quindi, la rete tende ad un attrattore determinato che appartiene alla sua superficie di stabilità.

- caso $w > 4$: in questo caso il valore di f' dipende dai parametri I , θ e naturalmente da y . Si può vedere che (vedi [Bee95]) esiste un intervallo per cui i punti di equilibrio diventano 3: 2 stabili e 1 instabile. Si può utilizzare l'esecuzione in macchina per mostrare i punti di equilibrio come in figura 3.10. O anche si può utilizzare il programma per cercare punti di equilibrio instabile. Interessante a tal proposito la figura 3.11. Se si osserva la conformazione della superficie di stabilità in questo caso con w fissato (vedi 3.12) è evidente come al

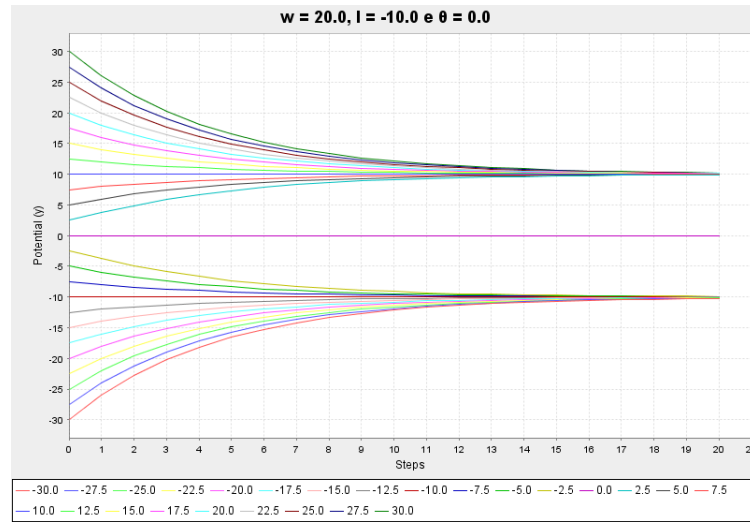


Figura 3.10: Comportamento dell'attività y del neurone nel tempo in funzione di diversi valori iniziali y_0 . In questo caso il peso dell'autoconnessione è $w = 20$, mentre gli altri parametri sono fissati con $\theta = 0$ e $I = -10$. È evidente come siamo in presenza di 3 punti di equilibrio. Uno centrale, di equilibrio instabile: se il valore iniziale è $y_0 = 0$, il valore dell'attivazione del neurone rimane 0. In tutti gli altri casi tende a due valori simmetrici rispetto a 0.

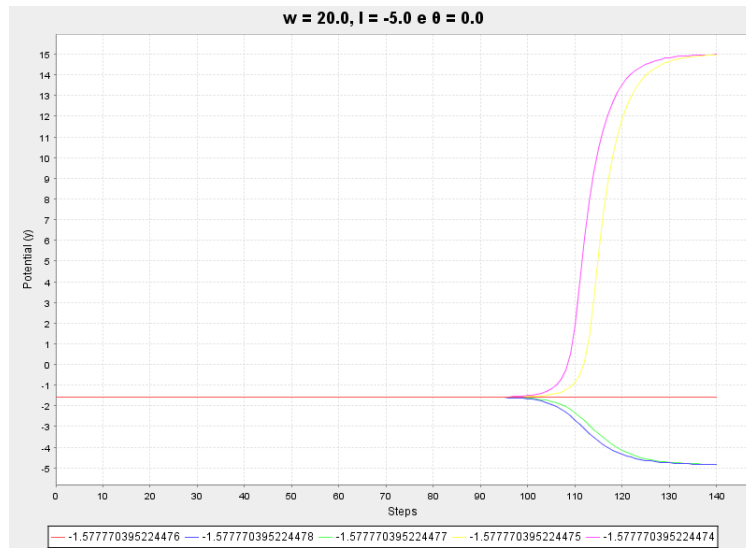


Figura 3.11: Un punto di equilibrio instabile trovato per via sperimentale. I parametri sono $w = 20$, $I = -5$ e $\theta = 0$. La risoluzione numerica dei *double* in questo caso ci permette di trovare il punto di instabilità con 16 cifre significative: il risultato è l'equivalente di una misura: possiamo dire che il punto di equilibrio instabile è il numero trovato con un errore sull'ultima cifra: $p_{equilibrio} = -1.577770395224476 \pm 0.0000000000000001$

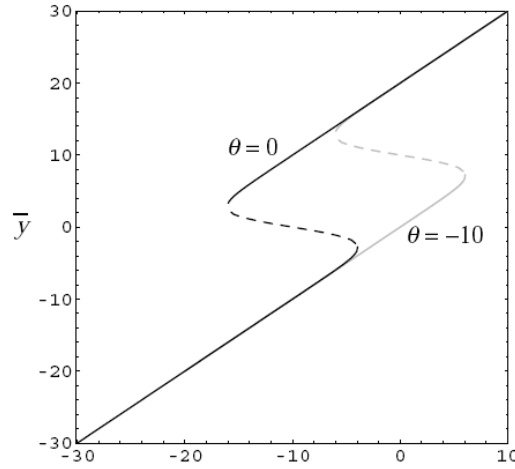


Figura 3.12: Superficie di stabilità per la rete CTRNN con un neurone.

variare di un altro parametro (l'input I) risulta esserci una *biforcazione pitchfork*. Infatti per valori di input inferiori a circa -16 esiste un unico punto di equilibrio stabile. Per valori superiori il punto cambia di stabilità diventando instabile e in più appaiono due nuovi punti di equilibrio stabile.

In ciascuno di questi esempi, è stata scelta come costante di tempo $\tau = 1$. Per avere una stabilità numerica assoluta della soluzione (ovvero per essere sicuri di trovare le stesse soluzioni della corrispondente equazione continua) occorre che sia soddisfatta la relazione 3.9. Quindi in questi esempi il passo di approssimazione è stato scelto pari a $\Delta t = 0.2$. Ma cosa accade se Δt si approssima al valore 2? La figura 3.13 mette in evidenza che in tal caso l'equazione simulata diviene instabile e oscillante. Quindi anche nel

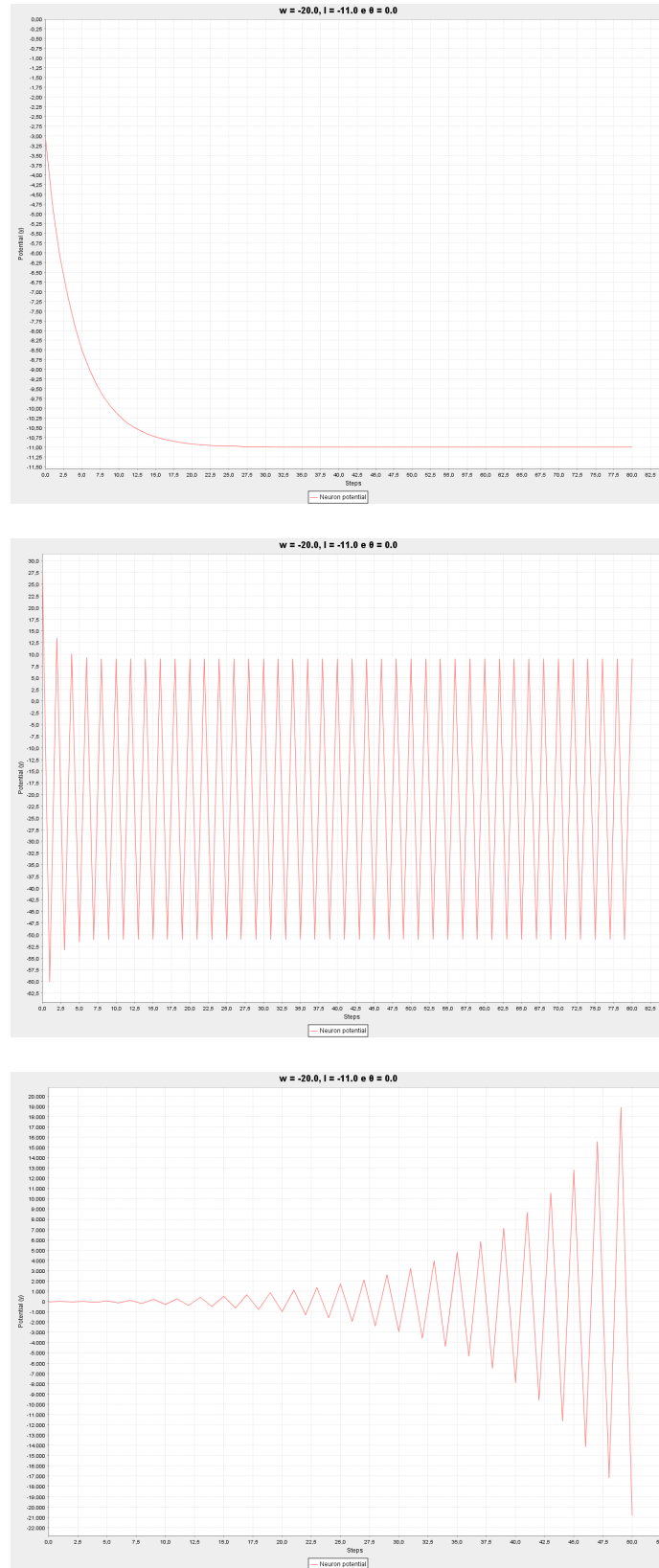


Figura 3.13: Tre figure che mostrano l'importanza del parametro Δt nell'approssimazione numerica. Tutte mostrano l'evoluzione temporale dell'attivazione del nodo. Nella prima figura a sinistra il parametro è posto a $\Delta t = 0.2$. Nel secondo a $\Delta t = 1.99$ e nel terzo $\Delta t = 2.1$.

prosieguo si sceglieranno valori del tipo:

$$\begin{aligned} 1 &< \tau_i < 70 \\ \Delta t &= 0.2 \end{aligned}$$

in modo da assicurarci che sia sempre verificata la relazione

$$\frac{\Delta t}{\tau_i} \ll 2$$

3.9 Ricerca dei parametri: Apprendimento

Stabilite le potenzialità e le capacità di rappresentazione di una rete neurale DTRNN è necessario dare delle procedure che formalizzino metodi di apprendimento per la rete stessa. Cosa si intende per apprendimento? Il sistema di equazioni che compongono una DTRNN dipende da diversi parametri, e al variare di tali parametri otteniamo sistemi dinamici diversi, con soluzioni e caratteristiche diverse. La procedura che chiamiamo apprendimento consiste in una *ricerca indirizzata* nello spazio dei possibili parametri allo scopo di trovarne opportuni che vadano a costituire equazioni con soluzioni dalle caratteristiche desiderate. Tra i metodi più usati per l'apprendimento in reti ricorrenti occorre citare la generalizzazione dell'algoritmo di apprendimento di propagazione all'indietro – *back-propagation* – a reti ricorrenti del tipo di Hopfield (vedi [Pin87]). L'algoritmo è stato ulteriormente generalizzato per reti ricorrenti generiche (vedi [Pea95]).

In pratica però questi algoritmi – Recurrent Back Propagation (RBP), Back Propagation Through Time (BPTT) – sono legati ad una preformulazione teorica del problema e non consentirebbero alla rete di adattarsi dinamicamente alle risposte dell’ambiente. Pertanto nel lavoro presente si è scelto, seguendo soprattutto le linee guida di [FM94], di effettuare una ricerca nello spazio dei parametri delle equazioni che compongono le DTRNN attraverso il metodo degli algoritmi genetici.

3.9.1 Algoritmi genetici

In breve si può descrivere un algoritmo genetico come una ricerca guidata all’interno dello spazio dei parametri. Tale metodo fu proposto da John Holland ([Hol75]) ed è una classe di *algoritmi di ricerca* che si ispirano, come evocato dal nome, ai meccanismi della *selezione naturale*. Essi simulano l’evoluzione di una popolazione di individui che rappresentano soluzioni candidate di uno specifico problema, “darwinianamente” favorendo la sopravvivenza e la riproduzione degli individui migliori. Alla base di tali algoritmi abbiamo fondamentalmente i seguenti elementi:

- Una codifica C tale che ad un dominio D di elementi che costituisce lo spazio degli individui (*fenotipi*) da valutare, associa stringhe di lunghezza definita r . L’insieme S formato dalle stringhe di lunghezza r possiede 2^r elementi ed è detto

Popolazione P dell'AG

1	0	1	1	1	0	1				1	0	0	genotipo 1
1	1	1	0	1	0	1				1	0	1	genotipo 2
1	0	0	1	1	0	1				0	0	0	genotipo 3
1	0	1	0	1	0	1				1	1	0	genotipo n

Figura 3.14: Codifica in n stringhe di una popolazione spazio di ricerca.

$$C : d \in D \rightarrow C(d) = s \in S$$

Tale codifica deve risultare *invertibile* di modo che ogni stringa $s \in S$, espressione della codifica C , si possa risalire al corrispondente individuo codificato $d \in D$ senza ambiguità. Per mezzo di tale funzione C un numero finito n di individui viene codificato a formare una *popolazione* P costituita da stringhe g_i dette *genotipi*

$$P \equiv \{g_1, \dots, g_n\}$$

- Una funzione f di valutazione degli individui detta funzione di *fitness* che associa ad ognuno di essi un valore.

$$f : d \in D \rightarrow f(d) = v \in R$$

L'insieme degli individui sarà selezionato in modo da mini-

mizzare (o massimizzare) i valori prodotti dalla fitness.

- Un numero finito K di operatori $\hat{O}_k$ con $k \in 1, \dots, K$ che agiscono su di un sottoinsieme di S di stringhe codificanti gli individui.

$$\hat{O}_k : \bar{S} = \{s_h\}_1^H \subset S \rightarrow \hat{O}_k(\bar{S}) = \{s_l\}_1^L \subset S$$

Dato il sottoinsieme G delle stringhe S

$$G = \{s \in S \mid C^{-1}(s) = d \in D\}$$

questi operatori hanno la proprietà di essere *chiusi* sull'insieme G , ovvero

$$\hat{O}_k(G) = G$$

ovvero l'azione di questi operatori su stringhe che codificano individui restituiscono ancora stringhe che codificano individui appartenenti al dominio D .

Il processo che porta alla scelta di individui più forti selezionati attraverso la fitness viene detto *evoluzione* e i passi di tale processo prendono appunto il nome di *algoritmo genetico*. Un algoritmo genetico (vedi algoritmo 1) consiste nel seguente processo:

- Si inizializza una popolazione $P(0)$ random al passo $t = 0$
- Fino ad una condizione di uscita, che può essere un certo valore della fitness o semplicemente un certo numero di cicli, si eseguono:

- valutazione della $P(t)$ al tempo t mediante la fitness
- applicazione degli operatori $\hat{O}_k$ su $P(t)$ fino a costituire una nuova popolazione $P(t+1)$

Gli operatori servono a perturbare la popolazione per effettuare una ricerca nello spazio dei possibili individui. Gli operatori classici sono di tre tipi

- operatore *selezione* $\hat{S}$,

$$\hat{S} : P = \{g_i\}_1^n \rightarrow \hat{S}(P) = \{\bar{g}_l\}_1^L$$

ovvero algoritmo di scelta degli individui all'interno della popolazione. La scelta naturalmente avviene in base alla valutazione della fitness; può avvenire in maniera probabilistica ed ha comunque lo scopo di selezionare gli individui più forti;

- operatore *crossover* $\hat{C}$,

$$\hat{C} : (g_i, g_j) \in G \times G \rightarrow \bar{g} \in G$$

il quale agisce su due individui *genitori* selezionati, e combina insieme due genotipi a formarne uno nuovo in maniera corretta, ovvero in modo che codifichi ancora un individuo della popolazione e sia ancora nel dominio della funzione di fitness;

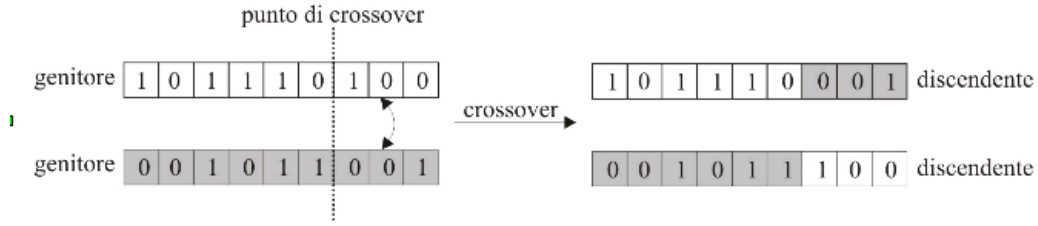


Figura 3.15: Schema dell'operatore di crossover

- operatore *mutazione* $\hat{M}$,

$$\hat{M} : \{g_l\}_1^L \in G \rightarrow \{\bar{g}_j\}_1^J \in G$$

il quale agisce probabilisticamente sui genotipi dell'insieme selezionando una porzione di stringa in modo da mutare in essa tutti gli 0 in 1 e viceversa.

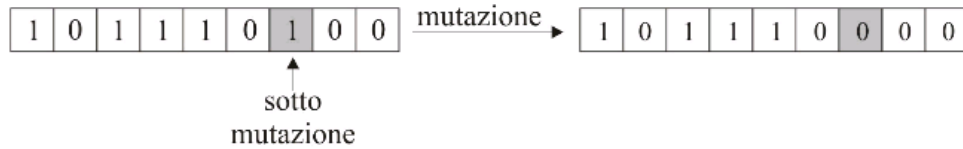


Figura 3.16: Schema dell'algoritmo di mutazione

Tale processo porta alla selezione di popolazioni con caratteristiche guidate dalla fitness. Ogni popolazione risultante da un ciclo è detta *generazione*. Nel caso in cui sulla stessa popolazione si eseguano diversi tipi di evoluzioni con fitness o operatori diversi, ognuna di queste evoluzioni sarà detta *epoca*. Un'epoca è dunque

Algorithm 1 Ciclo di un algoritmo genetico

AG

```

t=0
inizializza la popolazione P(t) in maniera casuale
v ← fitness ( P(t) )
while (! criterio_fermata)
    t ← t+1
    Mutazione ( Crossover ( Selezione ( P(t), v ) ) )
    v ← fitness ( P(t) )

```

un'unico processo di evoluzione con caratteristiche di operatori e fitness che rimangano immutate.

3.9.2 Codifica e scelta degli operatori

Per la codifica di reti CTRNN si sono seguite in buona parte le tecniche esposte in [PT04a, PT04b]. In particolare si è scelto di tenere fisso il *cablaggio* della rete: ovvero all'inizio dell'addestramento si è scelto di quanti elementi (nodi) doveva essere composta e come dovessero essere connessi fra loro. I parametri che vogliamo ricercare con l'algoritmo genetico sono i parametri dei neuroni, ovvero i pesi w , le costanti di tempo τ e le soglie θ . Fissato un intervallo predefinito all'interno del quale farli variare, la codifica del valore all'interno dell'intervallo è una codifica binaria con una risoluzione in K bit scelta a priori (per gli esperimenti si è fissato una risoluzione di 5 bit). Il risultato finale è che l'intera rete è

codificata con un'unica stringa di 0 e 1, con sottosequenze di K bit a codificare ciascun parametro. Fissato il numero di parametri da codificare per ciascuna rete, è predeterminata la posizione del parametro all'interno della stringa totale. Così si può risalire al valore di un parametro codificato dalla posizione m alla posizione $m + K - 1$ (estremi compresi) attraverso i seguenti passi:

- Calcolo del “valore binario” ovvero del valore compreso tra 0 e $2^K - 1$ tramite la formula:

$$p_{bin}^m = \sum_{i=1}^K bit_{i+m-1}^{K-i} \text{ con } bit_i \in \{0, 1\}$$

- Calcolo del “valore analogico”, conoscendo l'intervallo di variazione del parametro $I = (p_{min}^m, p_{max}^m)$ tramite la formula

$$p_{analog}^m = p_{min}^m + \frac{p_{bin}^m \cdot (p_{max}^m - p_{min}^m)}{2^K - 1}$$

Per quel che riguarda gli operatori si è fatta una scelta che semplifichi molto l'algoritmo. Il crossover, infatti, non è stato implementato, la selezione sceglie i primi B individui all'interno della popolazione e su di essi viene applicato l'algoritmo di mutazione. Questo algoritmo prevede una mutazione fissa in percentuale sull'intero genotipo, pertanto se si imposta una mutazione al 5%, e ipotizzando una stringa di 500 bit, verranno mutati 25 bit del genoma.

3.10 Esempio di apprendimento di attrattori

Diamo ora un esempio di applicazione del metodo di apprendimento che fa uso di algoritmi genetici appena esposto per far apprendere degli *attrattori* alla rete DTRNN descritta in precedenza. Sfruttando la definizione dei punti fissi stabili, facciamo apprendere ad una rete con un numero di neuroni sufficienti (in questo caso cinque) tre attrattori. Infatti con 5 neuroni posso avere al massimo una configurazione di parametri che mi permetta di avere 3^5 punti di equilibrio (vedi 3.6). Quindi ci aspettiamo che la configurazione con *almeno* 3 punti di equilibrio risulti sufficientemente probabile. Vengono selezionate 3 configurazioni della rete S_k , con $k \in 1, \dots, 3$ nelle quali l' i -esimo neurone sarà nello stato $\bar{x}_i^k$: queste configurazioni saranno i tre stati finali desiderati.

All'inizio di ogni generazione per ciascun attrattore ogni singola rete viene portata in uno stato iniziale che sia un intorno dell'attrattore⁹, si forza l'*evoluzione temporale* della rete (vedi 3.7), in questo caso è sufficiente un numero di step $n = 100$, e si misura la distanza euclidea dall'attrattore (presa con un segno negativo).

$$fitness^k = - \sum_{i=1}^5 (\bar{x}_i^k - x_i)^2 \text{ con } k \in 1, \dots, 3 \quad (3.11)$$

⁹Cerchiamo per la precisione punti fissi asintoticamente stabili del sistema dinamico individuato dalla DTRNN e sfruttiamo la proprietà di questi punti per cui posto il sistema in uno stato vicino ad uno di essi, la sua evoluzione dopo un tempo sufficientemente lungo lo porterà nel punto fisso suddetto.

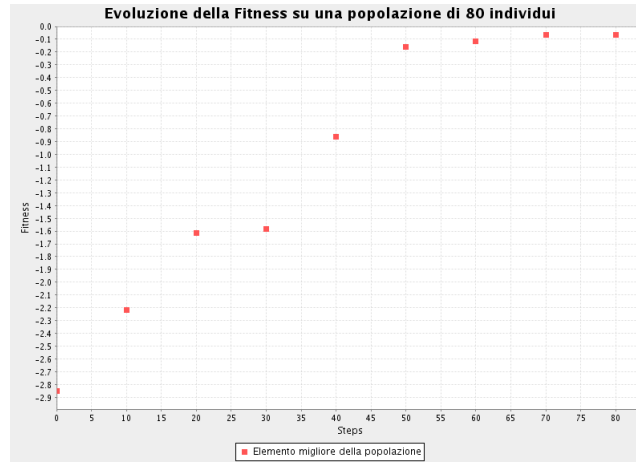
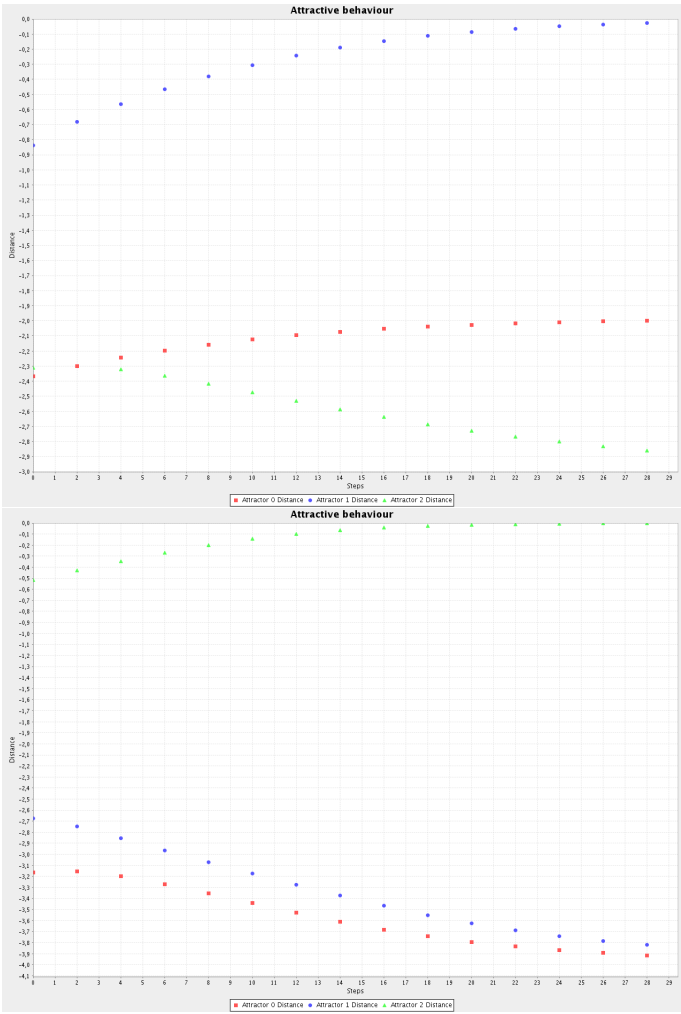


Figura 3.17: Evoluzione della media sulla popolazione della valutazione di fitness. In ascissa è riportato il numero della generazione. In ordinata il valore della valutazione.

di modo che la fitness totale sarà la somma delle tre:

$$F_{tot} = \sum_{k=1}^3 fitness^k$$

Le figure 3.18 mostrano che tale metodo raggiunge il risultato sperato. Per ogni generazione viene applicato come valutazione il metodo suddetto. Dopo 80 generazioni fatte *evolvere geneticamente* (secondo l'algoritmo esposto nel paragrafo precedente) su di una popolazione di 80 individui si può esibire una rete con le caratteristiche desiderate.



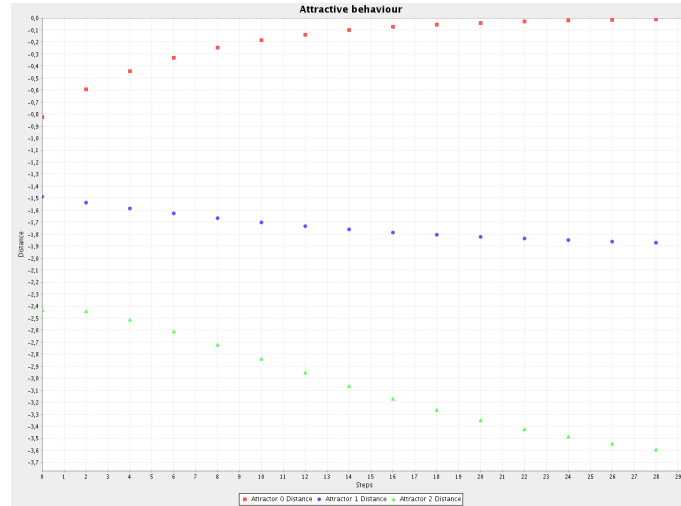


Figura 3.18: Apprendimento di tre attrattori. Le tre figure verificano che se la rete si trova in uno stato iniziale vicino a ciascuno degli attrattori che sono stati appresi, lo stato della rete dopo una breve *evoluzione temporale* della rete (dopo un numero di step $n = 30$) cade nell'attrattore stesso. In ascissa lo step n , in ordinata la distanza euclidea dall'attrattore invertita di segno (vedi 3.11). Per fissare le idee, nell'ultima delle tre figure, osserviamo che la rete allo step $n = 0$ viene posta in un intorno dell'attrattore S_1 . Ad ogni step di evoluzione la rete si avvicina allo stato S_1 fino a coincidere con esso (punti rossi) mentre si porta ad una distanza fissa dagli altri due attrattori S_2 ed S_3 (punti blu e verdi).

Capitolo 4

Gerarchie di reti

4.1 Reti come controllo di un agente

Allo scopo di controllare un agente la strategia usata è quella di sfruttare la dinamica delle reti CTRNN. Abbiamo considerato reti organizzate in maniera gerarchica con parte dei nodi messi in condivisione (nodi *common*): definiamo dunque *common* nodi che ricevono e da cui partono connessioni dirette verso più di una rete. Essi saranno indicati con una c in apice (ad esempio G^c). All'interno della rete abbiamo distinto tre tipi di nodi:

- nodi *output* O
- nodi *input* I
- nodi *generic* G

Tale suddivisione forma una partizione dell'intero insieme dei nodi della rete. I nodi output sono chiamati così perché il loro valore di uscita (se necessario opportunamente filtrato) è utilizzato per

controllare gli effettori dell'agente. L'attivazione dei nodi di input, invece, è determinata unicamente in base ai valori provenienti dai sensori dell'agente opportunamente mediati. I rimanenti sono detti nodi generici (*generic*).

Per formalizzare come l'agente interagisce con l'ambiente, un possibile algoritmo è il seguente:

- Al passo $n = n_0$ la rete è inizializzata con valori di attivazione casuale dei nodi. La rete quindi si troverà in uno stato $S = S(n_0)$.
- Finché non si verifica una condizione di uscita (che può essere ad esempio un predeterminato numero di passi n) vengono eseguiti i seguenti passi:
 - è determinato il valore dei nodi di input in base ai valori dei sensori.
 - la rete viene fatta evolvere per un certo numero di passi discreti M . M deve essere scelto in modo tale che lo stato della rete $S(n + M)$ sia prossimo ad un punto di equilibrio della rete, se esso esiste. Useremo la notazione $S(n + M) \approx S(+\infty)$ per esprimere questo concetto.
 - Il passo viene aggiornato a $n \leftarrow n + M$.
 - il valore dei nodi di output è inviato ad un “controller” che trasformi questi valori in opportuni segnali, per gli effettori.

In altre parole, dato un input sensoriale, stiamo supponendo che la dinamica della rete evolva sicuramente in uno stato asintoticamente stabile: le attivazioni degli output passate agli effettori saranno dunque espressione di tale stato di equilibrio stabile. Per gli esperimenti riguardanti più reti in cascata si è scelto di aggiornare tutte le reti in sincrono come se si trattasse un'unica rete. L'implementazione, comunque, è realizzata in modo da permettere di aggiornare le reti in maniera separata.

4.2 Rete di livello inferiore

4.2.1 Primitive motorie

L'oggetto del primo esperimento è rivolto allo sviluppo di una rete CTRNN che codifichi appropriati comandi in risposta alle sollecitazioni dell'ambiente esterno tali da consentire il governo di un agente. In particolare il tipo di *behaviour* che si vuole sviluppare è la capacità di muoversi all'interno di un ambiente evitando le pareti. Si è dunque costruita una rete CTRNN di sette nodi. Cinque di essi (per usare la terminologia di Tani - vedi [PT04a, PT04b] - li chiamiamo nodi "bottom") sono stati collegati ai nodi input. Di questi cinque, due inviano direttamente agli effettori (output, dunque). Questi cinque nodi inoltre sono completamente interconnessi tra loro. Ad essi aggiungiamo altri due nodi (nodi common) che non ricevono direttamente dai nodi input ma che sono connessi bidirezionalmente con i nodi bottom.

Il sensorio è costituito da 10 valori (i sonar, vedi paragrafo 4.2.4) che vanno a determinare le uscite dei 10 neuroni di input collegati ai bottom. Dunque in totale (vedi figura 4.1) abbiamo 10 *input* $\{I_k\}_1^{10}$, 2 *output* $\{O_{14}, O_{15}\}$ e 5 *generic* di cui due *common* $\{G_{11}, G_{12}, G_{13}, G_{16}^c, G_{17}^c\}$.

4.2.2 Studio del comportamento dinamico

L'interpretazione dell'addestramento e della "vita" del robot nel simulatore ha un'interpretazione dal punto di vista della teoria dei sistemi dinamici. Sappiamo che ad ogni nodo della rete corrisponde una legge di aggiornamento esprimibile tramite un'equazione differenziale, quindi l'intera rete di 7 nodi equivale ad un sistema dinamico di 7 equazioni differenziali:

$$\left\{ \begin{array}{l} \frac{dy_{11}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{11}, \{w_{11,j}\}_{j=1}^{N=17}, \theta_{11}, \{I_k\}_{k=1}^{K=10}) \\ \frac{dy_{12}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{12}, \{w_{12,j}\}_{j=1}^{N=17}, \theta_{12}, \{I_k\}_{k=1}^{K=10}) \\ \frac{dy_{13}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{13}, \{w_{13,j}\}_{j=1}^{N=17}, \theta_{13}, \{I_k\}_{k=1}^{K=10}) \\ \frac{dy_{14}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{14}, \{w_{14,j}\}_{j=1}^{N=17}, \theta_{14}, \{I_k\}_{k=1}^{K=10}) \\ \frac{dy_{15}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{15}, \{w_{15,j}\}_{j=1}^{N=17}, \theta_{15}, \{I_k\}_{k=1}^{K=10}) \\ \frac{dy_{16}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{16}, \{w_{16,j}\}_{j=11}^{N=17}, \theta_{16}) \\ \frac{dy_{17}}{dt} = f(\{y_j\}_{j=11}^{N=17}, \tau_{17}, \{w_{17,j}\}_{j=11}^{N=17}, \theta_{17}) \end{array} \right. \quad (4.1)$$

Questo sistema ha dunque 7 equazioni differenziali in 7 variabili ($\{y_i\}_{i=11}^{N=17}$), y_{14} ed y_{15} sono le attivazioni dei nodi di output, ovvero quelli collegati agli effettori, le ultime due invece sono le

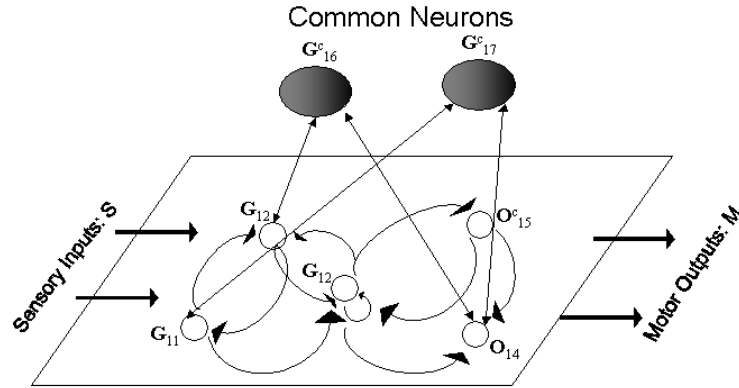
equazioni dei nodi common. L'unica prerogativa di questi due nodi, infatti è di non essere connessi direttamente con i nodi input. Queste equazioni dipendono da diversi parametri:

Tipo	θ_i	w_{ij}	w_{ik}	τ_i	I_k	Totale
Numero	7	49	50	7	10	123

Durante la fase di controllo, ovvero quando la rete viene usata come governo del robot, tutti i parametri sono bloccati tranne gli input I_i che corrispondono ai sensori. Lo scopo dell'addestramento è quindi la ricerca di una *famiglia di sistemi dinamici* tale che al variare dei parametri I_i faccia corrispondere un attrattore "adeguato" per il controllo dell'agente.

4.2.3 Codifica della rete

La rete precedentemente descritta è stata codificata in bit (vedi 3.9.2) per poter utilizzare un algoritmo genetico al fine di adeguare i parametri in modo che sviluppasse le caratteristiche desiderate. Per ogni nodo della rete ciascun parametro consisteva di un numero reale variabile all'interno di un intervallo predefinito. Si è scelta una risoluzione in bit per parametro di 5 bit di modo che possano assumere $2^5 = 32$ valori all'interno dell'intervallo. I parametri codificati per ogni nodo i sono le costanti di tempo τ_i , i bias θ_i , e i pesi entranti w_{ij} . Si è scelto arbitrariamente di far variare le costanti di tempo in un intervallo $[1, 70]$, i bias in un intervallo $[-1, 1]$ e i pesi in un intervallo $[-5, 5]$.

Figura 4.1: Struttura della rete di basso livello R_1

4.2.4 Simulazione

La rete viene addestrata adoperando un software *open source*, *Player/Stage/Gazebo* (vedi [KH04]), che possiede librerie in grado di simulare ambienti in 2D (*Stage*) e 3D (*Gazebo*), come pure l'interazione tra tali ambienti (o il mondo reale) e diversi tipi di robot (*Player*). Tra di essi si è scelto di governare un robot Pioneer 2DX, provvisto di sonar (per valutare le distanze dalle pareti). È possibile controllare il moto del robot attraverso due parametri: velocità lineare e velocità angolare (quest'ultima attraverso la rotazione della ruota perno che il Pioneer possiede). La rete effettua tale controllo attraverso i valori d'uscita dei due nodi di output: O_{14} è collegato alla velocità lineare mentre O_{15} alla velocità angolare. L'uscita di tali nodi è naturalmente traslata e ricalibrata sui valori di ingresso che accetta il Pioneer: così la velocità lineare, assumerà valori tra 0 e 200 mm/s (si è scelto di non includere velocità lineari negative: il robot potrà solamente andare

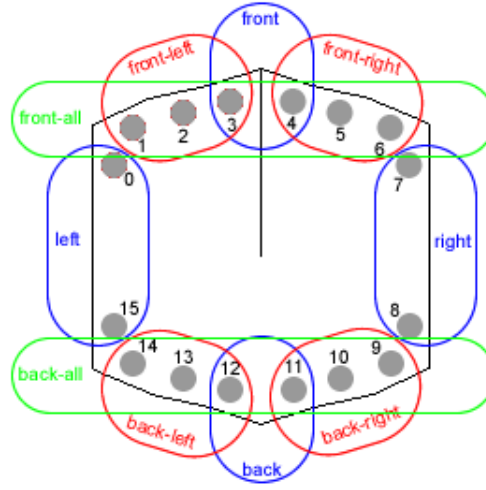


Figura 4.2: Disposizione dei sonar nel Pioneer 2DX

in avanti), mentre la velocità angolare varierà tra valori compresi tra -10 e 10 gradi/s. Il Pioneer ha in dotazione un totale di 16 sonar: 6 posizionati a “guardare” nella direzione di marcia, 6 rivolti all’indietro e 4 laterali. Alla rete per semplicità non vengono passati i sonar rivolti nella direzione opposta a quella di marcia: dunque i sonar input della rete sono in tutto 10 e li indichiamo con $\{S_i\}_1^{10}$. L’aggiornamento dei comandi con cui la rete controlla il robot avviene per intervalli di tempo discreti ΔT ad ogni istante $T_k = T_{k-1} + \Delta T$ con $k \in \{1, \dots, K\}$. Quindi al tempo $t = T_k$ alla rete vengono passati gli input ottenuti dall’interazione del robot con l’ambiente, cioè $S_i(T_k) \equiv S_i^k$ con $k \in \{1, \dots, K\}$ e $i \in \{1, \dots, 10\}$. In ciascun intervallo di tempo ΔT la rete si aggiorna *ad input fissi* come descritto nel paragrafo 4.1 avvicinandosi ad uno stato di equilibrio stabile, in modo tale che agli

effettori sia passata una buona approssimazione di una risposta asintotica della rete. Per essere un buon controllo, dunque, la rete dovrà essere simulata in modo da raggiungere comportamenti limite in un tempo piccolo rispetto all'interazione tra robot e ambiente. Per “vita” di un robot in un ambiente intendiamo, perciò, un certo numero di interazioni rete-robot, e la sua durata t_{life} è fissata da un numero $K \in \mathbb{N}$, di intervalli ΔT :

$$t_{life} = K\Delta T$$

Per questo esperimento sono stati utilizzati come parametri $T = 150ms$ e $K = 300$, quindi ogni simulazione in un ambiente aveva una durata di circa $t_{life} = 45$ secondi.

4.2.5 Valutazione della rete

Allo scopo di utilizzare l'algoritmo genetico descritto nel paragrafo 3.9.1, è stato necessario valutare il comportamento di un agente, controllato da una rete neurale nel modo precedentemente descritto, scegliendo un'opportuna funzione di fitness. Il punto di partenza di tale valutazione è la fitness di Floreano e Mondada (vedi [FM94]) che consta di tre elementi:

- un contributo che premia se il robot si tiene distante dal muro
- un contributo che premia il robot se va veloce
- un contributo che premia il robot se cambia poco direzione

Durante la “vita” del robot, intesa nel senso del paragrafo precedente, registriamo ad ogni T_k i valori dei sonar S_i^k , la velocità lineare V_L^k , e le velocità angolare V_A^k .

La fitness assume una forma del tipo:

$$F_{FM} = (1 - S_{max}) \cdot \bar{V}_L \cdot (1 - \sqrt{\bar{V}_A})$$

con

$$\begin{aligned} S_{max} &= \max \{S_i^k : k \in 1, \dots, K; i \in 1, \dots, 10\} \\ \bar{V}_L &= \frac{1}{K} \sum_{k=1}^K V_L^k \\ \bar{V}_A &= \frac{1}{K} \sum_{k=1}^K |V_A^k| \end{aligned}$$

S_{max} e V_L sono quantità normalizzate tra 0 e 1; V_A^k è ricalibrata in modo da assumere valori tra -1 e 1 : 0 indica nessuna velocità angolare. S_i^k è una misura di quanto il robot si sia tenuto vicino alle pareti: 1 rappresenta il caso in cui il robot abbia toccato le pareti. La fitness tiene conto del massimo S_{max} tra tutti i valori di sonar rilevati durante l'addestramento viene preso il più grande: questo nello spirito per cui basta una sola volta che il robot urti un ostacolo perché la rete venga scartata. Gli altri due parametri di cui teniamo conto sono il valore medio della velocità lineare ($\bar{V}_L$) e il valore medio del modulo della velocità angolare ($\bar{V}_A$) allo scopo di premiare reti che provocano velocità lineari alte e velocità

angolari basse (si prende la radice del valore di $\bar{V}_A$ in modo che tale parametro risulti meno importante al fine della valutazione).

4.2.6 Ambienti di addestramento

Affinché la fitness riesca a cogliere correttamente un buon comportamento dell'agente, uno dei problemi più ardui da affrontare è la scelta di buone condizioni di addestramento, al fine di poter scegliere dei parametri di rete *sufficientemente generali* da poter essere utili a governare il robot in ambienti che differiscono in maniera significativa da quello di addestramento. A tale scopo un suggerimento significativo lo si può derivare da [FM94], in cui le velocità iniziali che vengono passate al robot sono casuali, in modo che il robot debba recuperare da questo iniziale contributo casuale. Si è pensato, inoltre, ad un ulteriore espediente che ha provocato una diminuzione dei costi dell'addestramento (150 cicli sono sufficienti rispetto agli 800 dell'esperimento di [BF02]). L'idea è creare vari ambienti $\{A_i\}_1^L$ in cui il robot si trovi in situazioni "limite" per le quali si desidera che la rete apprenda una strategia di soluzione. Il robot ha tante vite quanti ambienti precostituiti. Si cerca una rete che tenga insieme i vari comportamenti nei diversi ambienti, pertanto la valutazione è il prodotto delle fitness f_{A_i} ottenute dalle varie vite nei diversi ambienti A_i :

$$F = \prod_{i=1}^L f_{A_i}$$

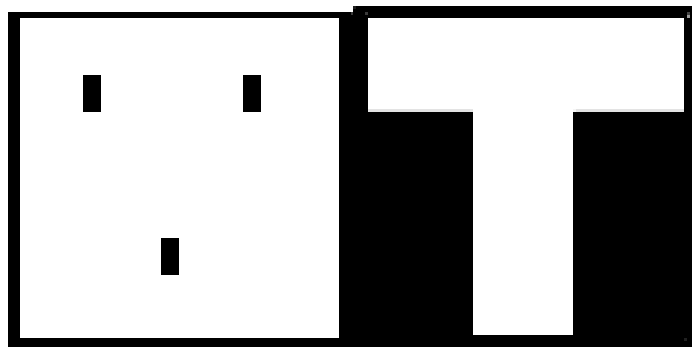


Figura 4.3: Ambienti di addestramento visti dall’alto: a sinistra stanza rettangolare con pochi ostacoli, a destra bivio a “T”.

In particolare per l’esperimento realizzato gli ambienti sono due e molto semplici: il primo un ambiente rettangolare con pochi ostacoli (vedi figura 4.3a) in cui il comportamento desiderato è che il robot impari a muoversi evitando gli ostacoli, il secondo un ambiente a “T” (vedi figura 4.3b) in cui il robot impari a seguire un corridoio e al bivio girare in una delle due direzioni possibile. Osserviamo che Floreano addestra la sua rete in un unico ambiente, mentre il presente approccio di *addestramento multiambiente* sembra più fruttuoso dal punto di vista della capacità di generalizzare che la rete acquista, e nel caso specifico per due ambienti, risulta anche in un abbattimento dei tempi di addestramento.

4.2.7 Rete Addestrata

Dopo circa 150 cicli di addestramento, le reti con fitness più alta mostrano di avere un buon comportamento negli ambienti in cui sono state addestrate. La rete migliore viene valutata anche

in ambienti diversi su cui non è stato effettuato addestramento. Durante ripetute esecuzioni su vari ambienti in simulazione la rete sembra aver *generalizzato* in maniera soddisfacente riuscendo a controllare il robot facendogli evitare con successo collisioni con ostacoli e tenendosi sufficientemente lontano dalle pareti. Nelle figure 4.4, 4.5 e 4.6 si mostrano alcune di queste prove. Per ciascun ambiente viene riportato un grafico di frequenza di occupazione del territorio: l'ambiente è suddiviso in piccole zone, e mentre il robot si muove, ogni $100ms$ viene registrata la sua posizione. I quadrati più scuri indicano che il robot ha trascorso più tempo nella zona. Il grafico evidenzia come il robot componga traiettorie opportune, percorrendo buona parte dell'ambiente senza fermarsi e senza urtare ostacoli. Anche in ambiente reale, nonostante i noti problemi di passaggio dalla simulazione al reale (vedi [LDW92, BP02]), il robot sembra riuscire ad avere un comportamento soddisfacente. Si è eseguita una semplice prova in un ambiente rettangolare privo di ostacoli, e il robot sembra riuscire ad evitare le pareti, pur “prendendo qualche rischio in più” rispetto alla simulazione (vedi figura 4.7).

4.2.8 Nodi Comuni

L'idea che sta alla base della costruzione della rete è che sia possibile influenzare il suo comportamento modificando i valori d'uscita dei nodi comuni. In tal modo se questi nodi comuni vengono collegati ad un'altra rete (rete di livello superiore), ovvero alla

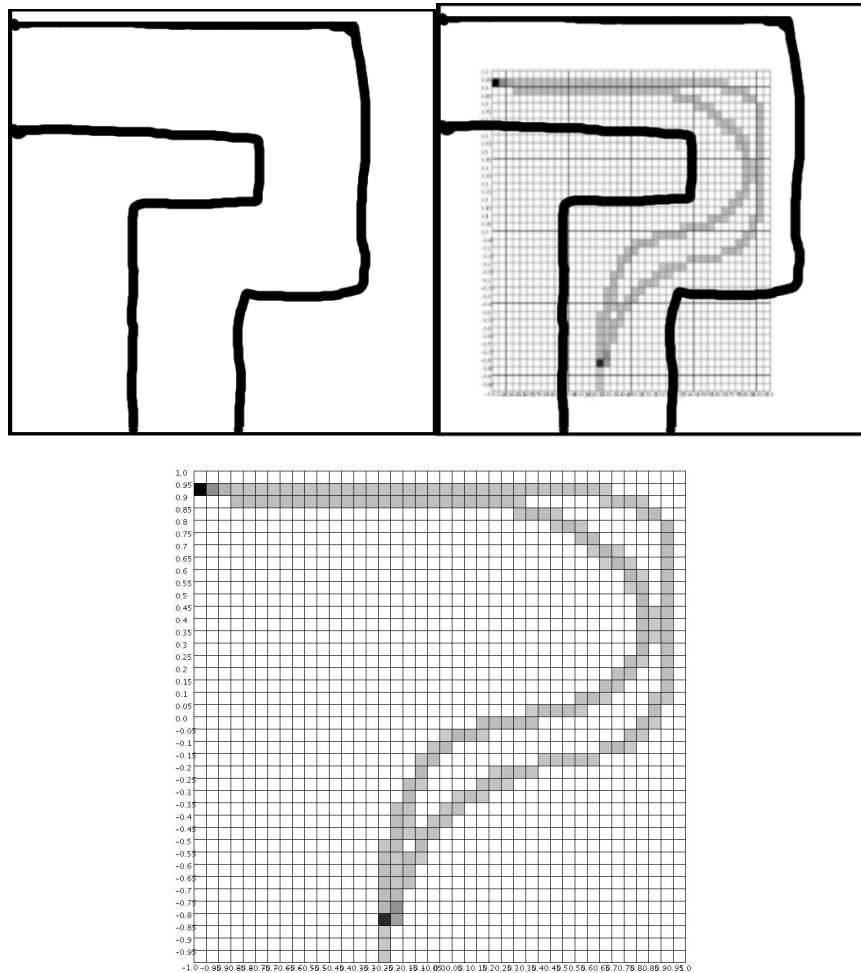


Figura 4.4: Uno degli ambienti simulati su cui è stata addestrata la nuova rete (in alto a sinistra) e frequenza delle posizioni occupate dopo circa 10 minuti di “vita” nell’ambiente (sotto). In alto a destra le due figure sono sovrapposte. Da notare i due quadrati che sono quelli più occupati: il robot arrivando davanti ad una parete chiusa gira su se stesso per poter tornare indietro occupando la zona per un tempo maggiore.

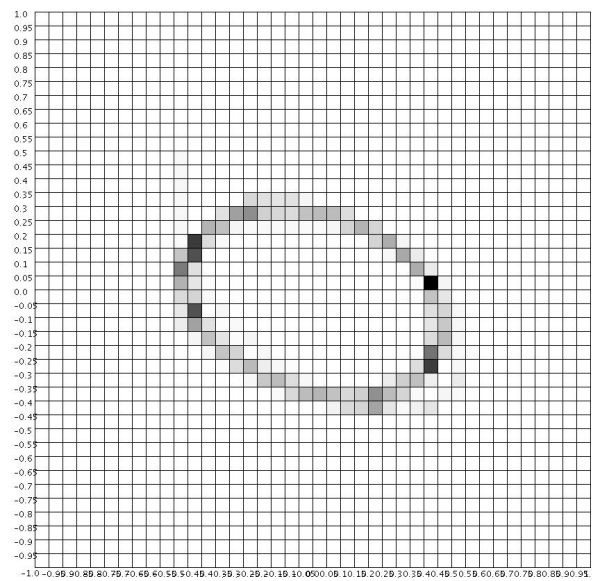
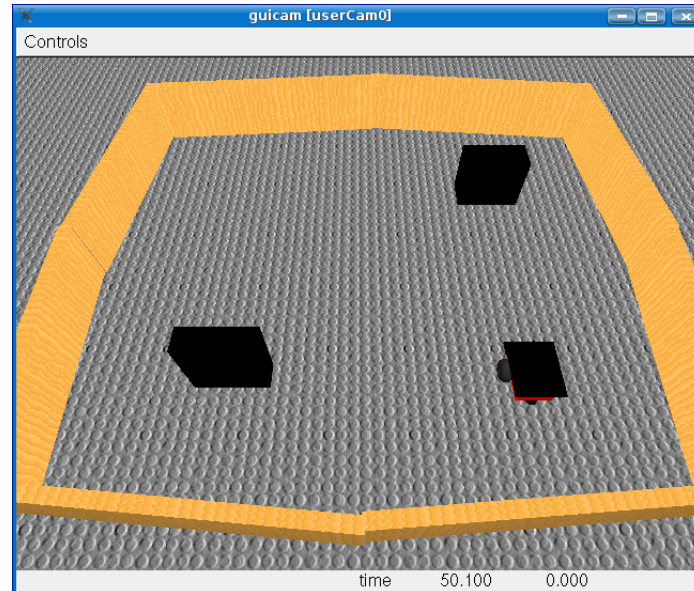


Figura 4.5: Esperimento in ambiente di simulazione 3D (*gazebo*) con un *pioneer2DX* che si muove in una stanza rettangolare con ostacoli (sopra). Sotto è mostrato il grafico di occupazione dell'ambiente.

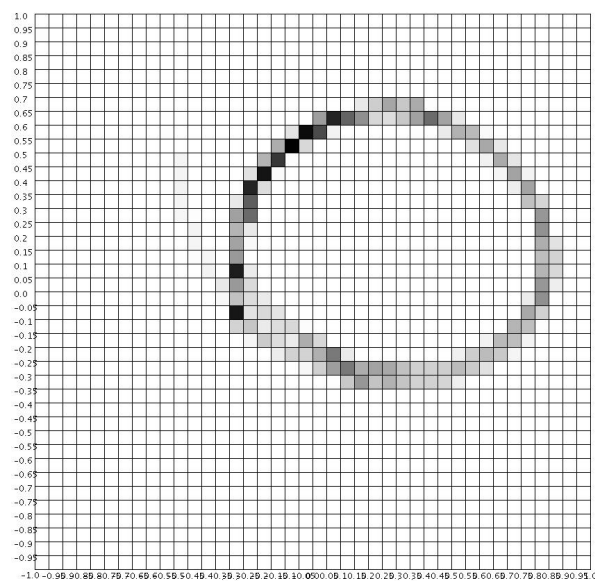
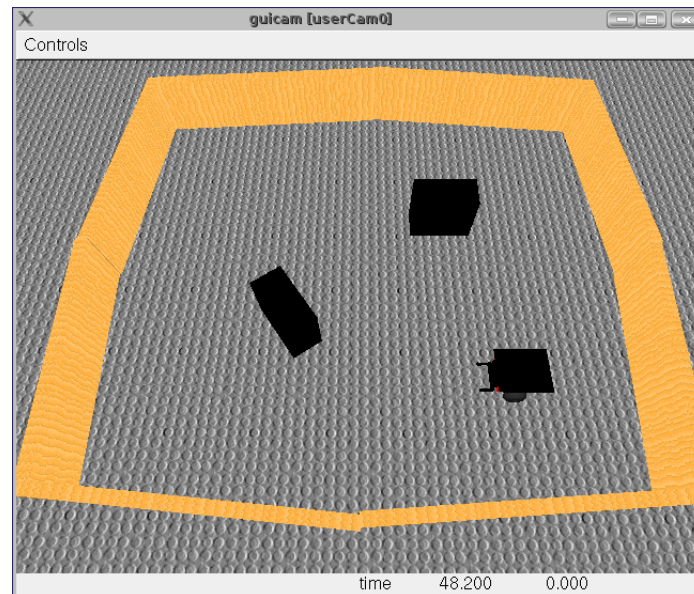


Figura 4.6: Altro esperimento in ambiente di simulazione 3D (*gazebo*) con un *pioneer2DX* che si muove in una stanza rettangolare con ostacoli in posizione diversa rispetto al precedente. L'ambiente è mostrato nella figura sopra, sotto ancora il grafico di occupazione.

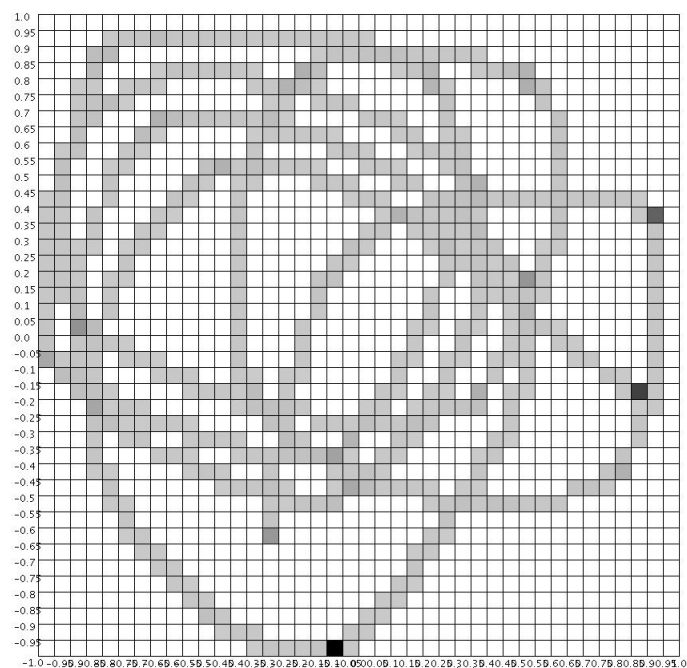


Figura 4.7: Frequenze di passaggio del robot in un *ambiente reale* rettangolare privo di ostacoli. I quadrati più scuri indicano che il robot ha occupato la zona un maggior numero di volte

legge di aggiornamento di ciascun nodo vengono aggiunti ulteriori ingressi provenienti da un'altra rete, la rete di livello inferiore è influenzata dagli stati della rete superiore. Per dare corpo a questa affermazione, facciamo variare i parametri di bias dei nodi comuni su di una rete già addestrata con la precedente fitness. L'idea è che variare i valori di bias è equivalente a variare valori di input ai nodi. Quindi successivamente quando a questi nodi saranno aggiunte connessioni da un'altra rete, sarà quest'ultima a far cambiare il comportamento della rete inferiore allo stesso modo.

Segmentiamo l'intervallo $(-1, 1)$ dei possibili valori di bias in 20 sotto-intervalli uguali U_i con $i \in \{1, \dots, 20\}$. Facciamo controllare un robot ad un bivio a "T" mediante reti preparate con i parametri fissati dall'addestramento, tranne che nei bias dei due nodi common, θ_{16} e θ_{17} , a cui facciamo assumere valori all'interno degli intervalli U_i . Dunque poniamo $\theta_{16} \in U_i$ e $\theta_{17} \in U_k$ per ogni $i, k \in \{1, \dots, 20\}$, per un totale di $20 \times 20 = 400$ esecuzioni. Si verifica (vedi figura 4.8) che attraverso opportuni valori di bias dei nodi common si può chiaramente influenzare il comportamento al bivio del robot: esiste un insieme di valori per cui il robot è forzato a girare a destra, ed un altro insieme per cui il robot è forzato a girare a sinistra. A commento di questo fenomeno Tani parla impropriamente di "emersione di una biforcazione parametrica". In realtà non si può parlare di biforcazione nel senso definito nel paragrafo 3.5.6, dato che non si verifica alcun cambiamento qualitativo nelle soluzioni delle equazioni 4.1. Comunque fissati gli

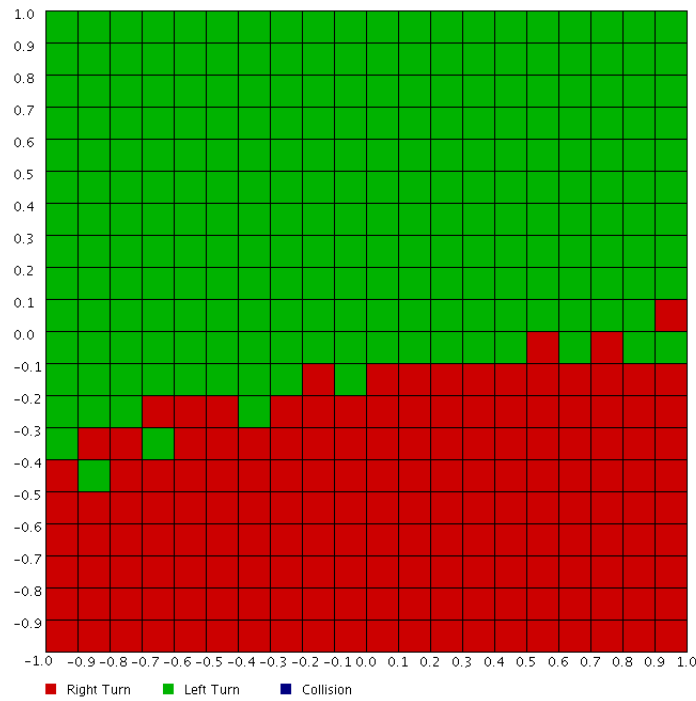


Figura 4.8: Nodi comuni. In ordinata ci sono i valori di bias il nodo common $G_{16}^{(c)}$, in ascissa ci sono quelli del nodo common $G_{17}^{(c)}$. Al variare di essi si registra sulla griglia il comportamento della rete: verde se il robot gira a sinistra, rosso se gira a destra.

input sensoriali esterni, la rete continua ad avere un unico attrattore e non si verificano situazioni in cui, ad esempio, al variare dei valori di bias, ci sia un cambiamento nel numero di attrattori, cosa che giustificherebbe l'uso della parola biforcazione. Più propriamente si può dire invece che variando i valori di bias della rete si ha un cambiamento di *behaviour* dell'agente all'interno della simulazione. Dunque possiamo affermare che la rete addestrata, che chiamiamo R_1 , ha *due comportamenti* (behaviour l e behaviour r) tra cui è possibile commutare attraverso l'influenza dei nodi common $G_{16}^{(c)}$ e $G_{17}^{(c)}$:

- behaviour l - andare avanti il più velocemente possibile, girando il meno possibile, stando lontano dalle pareti, e ad un bivio a T *girare a sinistra*;
- behaviour r - andare avanti il più velocemente possibile, girando il meno possibile, stando lontano dalle pareti, e ad un bivio a T *girare a destra*.

Una condizione equivalente alla variazione di bias si verifica se fissato il valore di bias aggiungiamo un input sui nodi common: dunque si verifica che *il comportamento della rete a parametri fissati (bias θ , pesi w e τ) risulta influenzabile tramite variazioni di input immessi sui due nodi common*.

Un'osservazione importante da fare è che variando le dimensioni del labirinto il grafico di figura 4.8 può variare di poco ma in maniera significativa la sua conformazione (vedi figura 4.9). Per

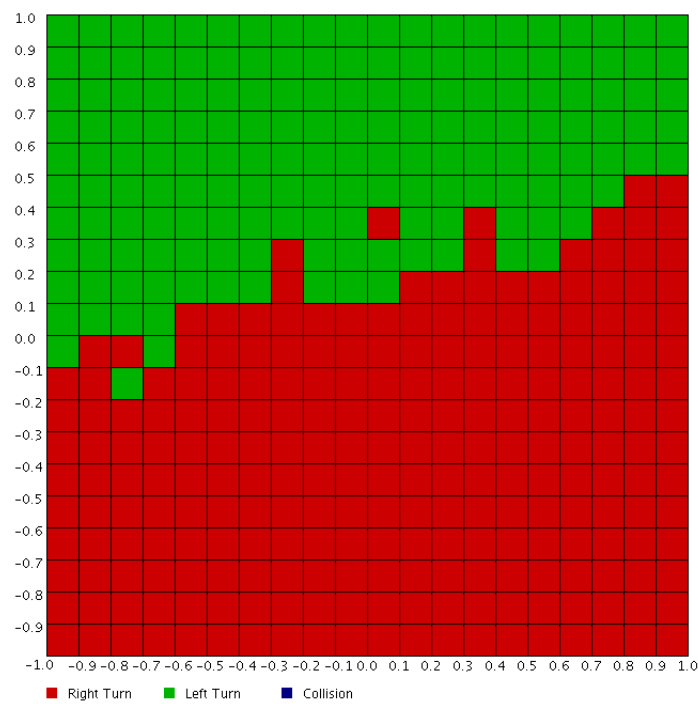


Figura 4.9: Comportamento del robot al variare dei bias dei nodi *common* per un labirinto di dimensione maggiore rispetto al grafico di figura 4.8.

essere sicuri di passare da un behaviour all'altro bisogna essere sicuri di forzare valori che siano in una zona indipendente dalla dimensione del labirinto (verso gli estremi alto e basso del grafico). Questa è una delle cause delle difficoltà di generalizzare ammessa da Tani nei suoi esperimenti: l'addestramento per la rete di livello superiore fu eseguita da Tani su labirinti di dimensione fissata, e la rete superiore non aveva modo di "apprendere" valori generali per forzare un cambiamento di direzione.

4.2.9 Estensione alla rete di livello inferiore

Alla luce del comportamento della rete R_1 addestrata e della verifica sperimentale su come è possibile influenzare la rete R_1 sembra chiaro dalla figura 4.8 che il nodo common G_{17}^c influenza meno il comportamento della rete rispetto a quanto fa il nodo common G_{16}^c . Quindi per semplificare il procedimento abbiamo fissato i valori di bias dei due nodi common, ma abbiamo posto un unico input I_{13} sul nodo G_{16}^c ampiamente sufficiente per poter controllare la rete. I valori di bias sono stati scelti come $\theta_{G_{16}^c} = -0.1$ e $\theta_{G_{17}^c} = -0.2$ in modo da trovarci circa nella zona di separazione di figura 4.8. Ci aspettiamo dunque che il cambiamento di behaviour avvenga dal passaggio di valori di input I_{13} da positivi a negativi. Infatti si verifica sperimentalmente che il cambiamento di comportamento per la rete addestrata si ha all'incirca attorno al valore 0.15. Pertanto la nuova rete R'_1 (vedi figura 4.10) si può costruire mediante due nodi: un nodo G_{19} con un autoconnes-

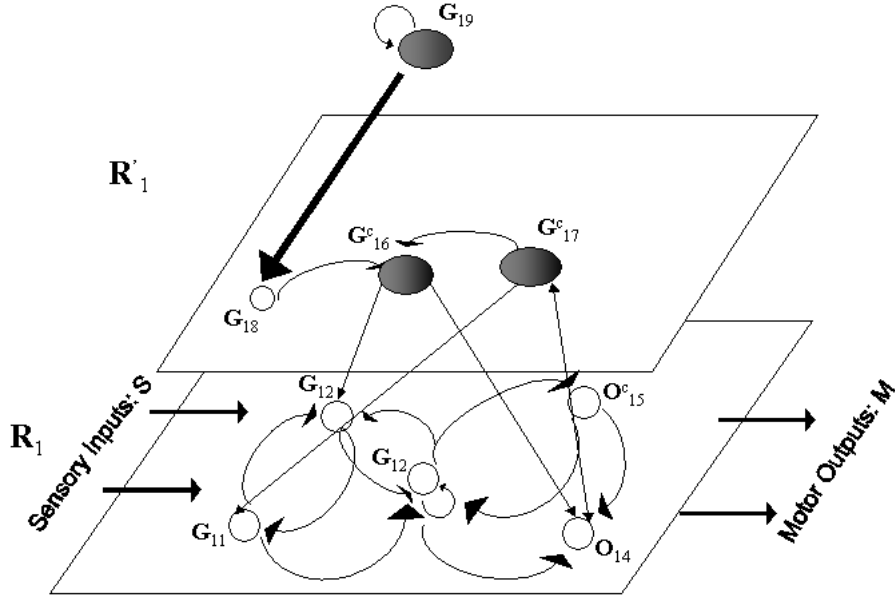


Figura 4.10: Rete $\bar{R}_1$ ottenuta aggiungendo in cascata la rete R'_1 alla rete R_1 di figura 4.1.

sione che ha un comportamento dinamico come quello studiato nel paragrafo 3.5.6, collegato ad un secondo nodo G_{18} che funziona semplicemente da filtro per l'input al nodo common G_{16}^c . Il nodo G_{19} è costruito con un peso di autoconnessione opportuno ($w > 4$) di modo che, come si è visto nel paragrafo 3.8.1, possenga una biforcazione al variare di un input in un determinato range di valori (vedi figura 4.13). In tal modo per un certo range di input al nodo G_{19} la rete inferiore è forzata sul behaviour r , per un altro range sul behaviour l e per un terzo range la scelta dipende dal valore del potenziale del nodo stesso. Con questa estensione alla rete di livello inferiore abbiamo un opportuno controllo del comportamento dei behaviour. Possiamo ridefinire come rete di

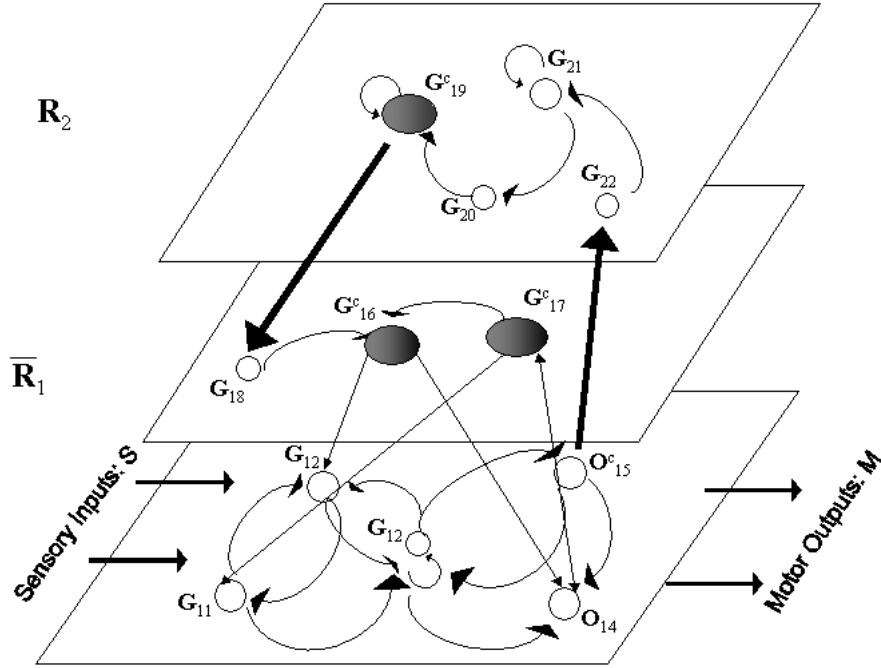


Figura 4.11: Aggiunta del secondo livello: la rete R_2 è posta in cascata sulla rete $\bar{R}_1$.

basso livello $_1$ la rete formata dall'unione della rete addestrata R_1 con l'aggiunta della parte qui costruita R'_1 :

$$\bar{R}_1 \equiv R_1 \cup R'_1$$

4.3 Rete di livello superiore

Con un ragionamento simile a quello usato nel paragrafo precedente, aggiungiamo un'ulteriore strato R_2 in grado di codificare due semplici piani

$$p_1 = (rl)^* \text{ e } p_2 = (lr)^*$$



Figura 4.12: Andamento dei valori d'uscita dei neuroni $O_{15}^c \in \bar{R}_1$ e $G_{21} \in R_2$ nell'ambiente: in ascissa ci sono i passi di integrazione, in ordinata i valori d'uscita dei neuroni. Dopo 1200 passi di integrazione il comportamento risultante è una sequenza rlr . Il neurone O_{15}^c è collegato all'effettore che controlla la velocità angolare. La rete $\bar{R}_1$ cade in numerosi stati di attrazione a seconda degli input sensoriali che le consentono di guidare l'agente senza urtare le pareti. La rete R_2 cade in due diversi attrattori a seconda del behaviour forzato sulla rete $\bar{R}_1$. Dalla figura si vede come per i primi 200 passi, la rete R_2 imposta una svolta a destra (il neurone G_{21} è in uno stato circa uguale a 1). Dopo che è stata effettuata la svolta la rete R_2 imposta il behaviour l (il neurone G_{21} è in uno stato circa uguale a 0). L'andamento prosegue identico per le successive svolte incontrate dall'agente durante l'esecuzione nell'ambiente.

ovvero *sequenze infinite di svolte destra-sinistra o sinistra-destra*. Per impostare il cambiamento di piano si usa informazione proveniente dalla rete di basso livello $\bar{R}_1$, in particolare dal nodo di output O_{15}^c (che pertanto diviene anch'esso un nodo common) il cui valore d'uscita è utilizzato per controllare la velocità angolare del Pioneer. Tale informazione può essere interpretata come una sorta di *propriocezione* (vedi nota 7 del capitolo 2) per il sistema. Dunque la rete R_2 è costruita con un nodo G_{22} che riceve la connessione dal nodo O_{15}^c e che funziona da filtro per l'informazione proveniente dal sistema di basso livello. Il nodo G_{22} invia ad un nodo G_{21} che ha un'autoconnessione ed è del tutto analogo al nodo G_{19} descritto precedentemente; allo stesso modo, infatti, si sono scelti parametri tali che gli consentano di possedere due zone di attrazione tra le quali è possibile scambiare a seconda dell'input, in questo caso appunto *l'uscita del nodo* O_{15}^c . Il nodo G_{21} invia ad un nodo G_{20} che ancora filtra l'informazione del nodo precedente per impostare l'opportuno behaviour con una connessione sul nodo G_{19} . A seconda dei due stati possibili del nodo G_{21} , il nodo G_{20} forza un input sul nodo G_{19} nel range del behaviour r o del behaviour l . Quando l'agente gira ad un bivio, il valore d'uscita del nodo O_{15}^c si sposta verso 0 o 1 a seconda che la svolta sia a destra o a sinistra. Al di sopra di una certa soglia ciò comporta il cambiamento di stato del nodo G_{21} che forza come “prossima svolta” quella opposta all'ultima avvenuta. Il risultato è una sorta di *ciclo di isteresi* tra due attrattori (vedi 4.13) che consente

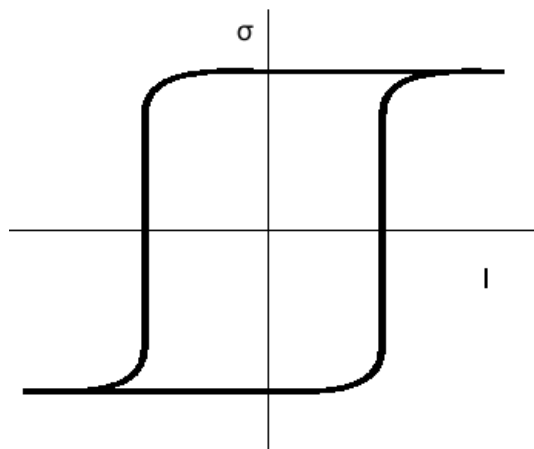


Figura 4.13: Attività di una rete CTRNN composta di un nodo con un'autoconnessione di peso $w > 4$ al variare dell'input esterno. In ordinata abbiamo il valore d'uscita del neurone, in ascissa l'input I

appunto l'attuazione dei piani p_1 e p_2 . Il sistema codifica il piano p_1 o p_2 a seconda dello stato iniziale del nodo G_{21} .

Questo approccio, confrontato con quello effettuato nell'esperimento di Tani, comporta degli evidenti vantaggi. La rete di livello superiore di Tani, memorizza durante l'addestramento un'evoluzione temporale, quindi una funzione del tempo che permetta, attraverso i valori assunti, di impostare il cambio di direzione al momento opportuno. Va da sé che, come lo stesso Tani verifica, cambiamenti nella lunghezza e nella dimensione del labirinto comportano errori nell'esecuzione delle sequenze perché occorrerebbero “tempistiche” diverse per adattarsi alle nuove proprietà dell'ambiente. Invece l'approccio *proprioceettivo* qui descritto, sfrutta l'informazione che il sistema ha su “quando sa di aver girato” allo scopo di impostare

il behaviour successivo. In tal modo la rete così costruita riesce bene a codificare un piano sufficientemente generale tale da poter essere eseguito per dimensioni e lunghezze di labirinto arbitrarie.

4.4 Conclusioni e possibili sviluppi

In primo luogo, il primo risultato ottenuto è una rete di basso livello con una capacità di generalizzazione ampia rispetto a quelle addestrate da Floreano e da Tani nei loro esperimenti. Tale capacità è stata ottenuta sfruttando un tipo di addestramento multiambiente che oltre a risultare in un apprendimento sensibilmente più veloce suggerisce pure la possibilità di poter opportunamente memorizzare più funzionalità contemporanee in una stessa rete attraverso l'opportuna scelta di ambienti adatti a tale scopo.

Si è verificato, inoltre, come un semplice piano p possa essere codificato attraverso una gerarchia di reti in cascata. Nei suoi lavori Tani era riuscito a far eseguire al robot delle sequenze *sinistra-destra* forzate da una rete di alto livello ma il suo approccio recava con sé dei problemi. Infatti egli stesso osservava a commento degli esperimenti che cambiando dimensione e lunghezza dei labirinti rispetto all'ambiente di addestramento il robot non era più in grado di compiere la sequenza giusta. Questo sostanzialmente per due motivi concorrenti:

- quelle memorizzate nella rete di alto livello di Tani erano funzioni temporali addestrate in maniera tale da forzare op-

portuni cambiamenti nella rete sottostante durante la sua evoluzione dinamica. Va da sé che un labirinto più lungo, richiede una diversa funzione del tempo e comporta un errore nella sequenza forzata sulla rete di basso livello.

- Inoltre come si evince nel paragrafo 4.2.8, a seconda della larghezza del bivio ci sono diversi valori di bias che possono dare risultati ambigui, dunque bisogna scegliere valori che siano in una zona di intersezione delle possibili svolte a destra (in basso nel grafico) e possibili svolte a sinistra (in alto nel grafico) per essere sicuro che tutto funzioni. Se si addestra la rete superiore su di un'unica dimensione di labirinto ci si deve aspettare che tale rete apprenda valori per forzare la rete sottostante che non siano “universali” per dimensione di labirinto.

L'approccio qui fornito invece consente di superare il secondo problema scegliendo “ad hoc” valori di input giusti su cui forzare la rete per essere sicuri di avere “svolta a destra” e “svolta a sinistra” tramite l'estensione R'_1 alla prima rete addestrata R_1 . Inoltre per il modo in cui è stata costruita la rete di alto livello R_2 , il primo problema non sussiste perché il piano codificato è svincolato dal tempo: è lo stesso sistema ad “accorgersi” di stare girando (tramite la connessione del nodo velocità angolare) in una sorta di *propriocezione*, e solo in seguito ad essa imposta il successivo behaviour al prossimo bivio. Naturalmente mancando altri dati sensoriali riguardanti la conformazione del labirinto, il sistema

non si accorge se sta realmente girando ad un bivio a “T” oppure se, ad esempio, sta ruotando attorno ad un ostacolo. Un passo successivo potrebbe essere aggiungere una rete che tramite ulteriori informazioni sensoriali (anche gli stessi sonar) “riconosca” di trovarsi ad un bivio, per effettuare il cambiamento di programma solo in questo caso.

Ancora, nei suoi esperimenti, Tani non riesce a trovare una rete di alto livello, attraverso l’addestramento tramite algoritmi genetici, che codifichi *tutte* le sequenze volute. L’approccio costruttivo mostrato all’interno del presente lavoro può suggerire la creazione di sequenze arbitrarie mediante la composizione di piccole reti dalla dinamica nota.

Una direzione in cui investigare è il metodo di costruzione della rete di basso livello. Nella rete dell’esperimento, come si è visto, “emergono” sperimentalmente due behaviour differenti influenzabili attraverso input su due dei nodi che la costituiscono. Sarebbe interessante trovare regole generali per la costruzione di tali reti, con relazioni quantitative sul numero di behaviour memorizzabili e il numero di nodi necessari per controllarli.

Appendice A

Stabilità Numerica

A.1 Metodi di integrazione numerica

Poniamo di voler risolvere in maniera approssimata il sistema di equazioni differenziale

$$\frac{d\mathbf{y}}{dt} = f(\mathbf{y}, \mathbf{t})$$

La maggior parte dei metodi di approssimazione si basano sulla *discretizzazione* delle equazioni. I metodi più potenti a tale scopo vanno sotto il nome di *metodi multi-step lineari*. Supponiamo di avere il sistema 3.6 con le condizioni di Cauchy soddisfatte. Sappiamo quindi che esiste un'unica soluzione $\mathbf{y}(t)$ nell'intervallo $[0, T]$ che raramente è possibile trovare in maniera *analitica*. Preso un numero reale, un *passo* $\Delta t > 0$ possiamo suddividere l'intervallo temporale in modo da avere:

$$t_n = n\Delta t \tag{A.1}$$

Lo scopo di una soluzione numerica è quello di trovare una sequenza $\mathbf{y}^0, \mathbf{y}^1, \dots$ tale che:

$$\mathbf{y}^n = \mathbf{y}(t_n)$$

In modo da ottenere un'equazione all' n -esimo passo approssimata della funzione in oggetto

La formula generica con s passi di approssimazione è data dalla *formula multi-step lineare ad s -passi*:

$$\sum_{j=0}^s a_j \mathbf{y}^{n+j} = \Delta t \sum_{j=0}^s b_j f(\mathbf{y}^{n+j}, t_{n+j}) \quad (\text{A.2})$$

A partire da questa formula si possono definire i polinomi caratteristici (operatori) $\rho(z) = \sum_{j=0}^s a_j z^j$ e $\sigma(z) = \sum_{j=0}^s b_j z^j$. Inoltre definiamo un *operatore di shift temporale* $\hat{Z}z^n = z^{n+1}$ nel caso discreto e $\hat{Z}z(t) = z(t + \Delta t)$ nel caso continuo. Possiamo dunque scrivere:

$$\begin{aligned} \sum_{j=0}^s a_j \mathbf{y}^{n+j} &= \sum_{j=0}^s a_j \hat{Z}^j \mathbf{y}^n = \rho(\hat{Z}) \mathbf{y}^n \quad \text{e} \\ \sum_{j=0}^s b_j f(\mathbf{y}^{n+j}, t_{n+j}) &= \sum_{j=0}^s b_j \hat{Z}^j f(\mathbf{y}^n, t_n) = \sigma(\hat{Z}) f(\mathbf{y}^n, t_n) \end{aligned}$$

di modo che la A.2 diviene:

$$\rho(\hat{Z}) \mathbf{y}^n - \Delta t \sigma(\hat{Z}) f(\mathbf{y}^n, t_n) = 0 \quad (\text{A.3})$$

A questo punto possiamo introdurre le nozioni di stabilità numerica che risultano in condizioni sulle caratteristiche della forma lineare multi-step ad s passi.

A.2 Stabilità e Stabilità assoluta numerica

La prima nozione ha a che fare con il comportamento dell'approssimazione per $\Delta t \rightarrow 0$: in questo caso la A.3 diviene semplicemente $\rho(\hat{Z})\mathbf{y}^n = 0$.

Definizione A.1 *Una formula lineare multi-step si dice **stabile** se tutte le soluzioni $\{\bar{\mathbf{y}}^n\}$ di $\rho(\hat{Z})\mathbf{y}^n = 0$ risultano essere limitate per $n \rightarrow \infty$, ovvero:*

Teorema A.1 *Una formula lineare multi-step è stabile se e solo se le radici del polinomio associato $\rho(z)$ soddisfano la relazione $|z| \leq 1$, ed ogni radice $|z| = 1$ risulta semplice (molteplicità 1).*

L'altra definizione di stabilità la si produce in relazione al passo temporale e pertanto si parla di *stabilità assoluta* o *stabilità debole*, per un particolare valore $k = \alpha\Delta t$. In questo caso supponiamo che la funzione f sia del tipo $f(\mathbf{y}, t) = \alpha\mathbf{y}$ con $\alpha \in \mathbb{C}$ e quindi prendiamo in considerazione un particolare tipo di equazione detta *equazione modello*:

$$\frac{d\mathbf{y}}{dt} = \alpha\mathbf{y} \tag{A.4}$$

Si può dimostrare che un sistema di ODE non lineare si può ridurre ad un insieme di problemi della forma dell'equazione: $d\mathbf{y}/dt = \alpha\mathbf{y}$. In questo caso la A.3 diviene $\left[\rho(\hat{Z}) - k\sigma(\hat{Z})\right] \mathbf{y}^n = 0$ da cui definiamo:

Definizione A.2 *Una formula lineare multi-step per un particolare valore $k = \alpha\Delta t$ è **assolutamente stabile** se tutte le soluzioni di $\left[\rho(\hat{Z}) - k\sigma(\hat{Z})\right] \mathbf{y}^n = 0$ risultano limitate per $n \rightarrow \infty$.*

Teorema A.2 *Una formula lineare multi-step è assolutamente stabile per un particolare valore $k = \alpha\Delta t$ se e solo se tutte le soluzioni del polinomio associato $[\pi_k(z) = \rho(z) - k\sigma(z)] \mathbf{y}^n = 0$ soddisfano la relazione $|z| \leq 1$, ed ogni radice $|z| = 1$ risulta semplice (molteplicità 1).*

Definizione A.3 *Si definisce **regione di stabilità** S di una formula lineare multi-step, l'insieme di tutti i $k \in \mathbb{C}$ per cui la formula risulta assolutamente stabile.*

Da quest'ultima definizione possiamo dedurre che una formula multi-step è stabile se e solo se il punto 0 appartiene alla sua regione di stabilità.

A.3 Metodo Forward di Eulero

Il *metodo Forward di Eulero* è il metodo di approssimazione numerica più semplice e consiste nell'approssimare la funzione ad og-

ni passo con il suo sviluppo in serie di Taylor arrestato al prim'ordine. Applichiamo dunque il primo passo di approssimazione alla 3.5:

$$y^1 = y(t_0 + \Delta t) = y(t_0) + \Delta t \left| \frac{dy(t)}{dt} \right|_{t_0} + \frac{\Delta t^2}{2} \left| \frac{d^2 y(t^*)}{dt^2} \right|_{t_0}$$

da cui otteniamo, escludendo il resto $R(\Theta(\Delta t^2))$

$$y^1 = y^0 + \Delta t f(y^0, t_0)$$

Proseguendo per un generico passo si può generalizzare ed ottenere la *formula multi-step per il metodo Forward di Eulero*:

$$y^{n+1} = y^n + \Delta t f(y^n, t_n) \quad (\text{A.5})$$

Confrontando la A.5 con la A.2, abbiamo i valori particolari:

$$s = 1, \quad a_0 = -1 \quad a_1 = 1 \text{ e } b_0 = 1$$

mentre tutti gli altri termini sono nulli. Possiamo calcolarci i polinomi associati:

$$\rho(z) = z - 1 \quad \text{e} \quad \sigma(z) = 1$$

Per il teorema A.1 la formula A.5 risulta stabile ($\rho(z)$ ha un'unica radice, $z = 1$). Cerchiamo i punti di assoluta stabilità attraverso le radici del polinomio $[\pi_k(z) = \rho(z) - k\sigma(z)] \mathbf{y}^n = 0$. Le

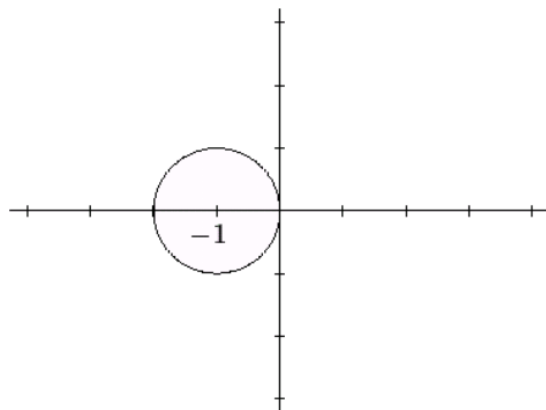


Figura A.1: Superficie di stabilità per il metodo Forward di Eulero.

soluzioni cercate sono quelle per cui:

$$|z| = |k + 1| \leq 1 \quad \text{con} \quad k \in \mathbb{C} \quad (\text{A.6})$$

Ne consegue che la regione di stabilità risulta essere un cerchio di centro -1 e raggio 1.

Bibliografia

- [Bee95] Randall D. Beer. On the dynamics of small continuous-time recurrent neural networks. *Adaptive Behavior*, 3(4):469–509, 1995.
- [BF02] Jesper Blynel and Dario Floreano. Levels of dynamics and adaptive behavior in evolutionary neural controllers. In *ICSAB: Proceedings of the seventh international conference on simulation of adaptive behavior on From animals to animats*, pages 272–281, Cambridge, MA, USA, 2002. MIT Press.
- [BP02] Dirk Bank and Erwin Prassler. Safe navigation of autonomous mobile systems by wide-range ultrasonic environment coverage. In *Proceedings of the IARP / IEEE-RAS Joint Workshop on Technical Challenge for Dependable Robots in Human Environments*, pages 42–49, Oktober 7-8 2002.
- [DTL⁺03a] Edoardo Datteri, Giancarlo Teti, Cecilia Laschi, Guglielmo Tamburrini, Paolo Dario, and Eugenio Guglielmelli. Expected perception: an anticipation-

- based perception-action scheme in robots. In *2003 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 934–939, 2003.
- [DTL⁺03b] Edoardo Datteri, Giancarlo Teti, Cecilia Laschi, Guglielmo Tamburrini, Paolo Dario, and Eugenio Guglielmelli. xpected perception in robots: a biologically driven perception-action scheme. In *11th International Conference on Advanced Robotics*, pages 1405–1410, 2003.
- [FA98] Andrew H. Fagg and Michael A. Arbib. Modeling parietal–premotor interactions in primate control of grasping. *Neural Networks*, 11(7-8):1277–1303, 1998.
- [FGB⁺01] Luciano Fogassi, Vittorio Gallese, Giovanni Buccino, Laila Craighero, Luciano Fadiga, and Giacomo Rizzolatti. Cortical mechanism for the visual guidance of hand grasping movements in the monkey - a reversible inactivation study. *Brain*, 124:571–586, March 2001.
- [FM94] Dario Floreano and Francesco Mondada. Automatic creation of an autonomous agent: Genetic evolution of a neural-network driven robot. In *Proceedings of the Conference on Simulation of Adaptive Behavior*,

- pages 421–430, Cambridge, MA, USA, 1994. MIT Press.
- [GFFR96] Vittorio Gallese, Luciano Fadiga, Leonardo Fogassi, and Giacomo Rizzolatti. Action recognition in the premotor cortex. *Brain*, 119:593–609, April 1996.
- [GG55] James J. Gibson and Eleanor J. Gibson. Perceptual learning: Differentiation or enrichment? *Psychological Review*, 62:32–41, 1955.
- [GH86] John Guckenheimer and Philip Holmes. *Nonlinear Oscillations, Dynamical Systems, and Bifurcations of Vector Fields*. Springer-Verlag, 1986.
- [Gib66] James J. Gibson. *The Senses Considered as Perceptual Systems*. HtnMif, Boston, 1966.
- [GM92] Melvyn A. Goodale and A. David Milner. Separate visual pathways for perception and action. *Trends in Neuroscience*, 15:20–25, 1992.
- [GMLK00] Stefan Geyer, Massimo Matelli, Giuseppe Luppino, and Zilles Karl. Functional neuroanatomy of the primate isocortical motor system. *Anatomy and Embryology*, 202(6):443–474, November 2000.
- [HC95] Michael Hines and Nicholas T. Carnevale. Computer modeling methods for neurons. *In: The Hand-*

book of Brain Theory and Neural Networks, edited by M.A. Arbib, pages 226–230, 1995.

- [HH52] A. L. Hodgkin and A. F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiol*, 117:500–544, 1952.
- [Hol75] John H. Holland. *Adaptation in natural and artificial systems*. University of Michigan Press, Ann Arbor, MI, 1975.
- [JM05] Chris Jones and Maja J. Mataric. *Autonomous Mobile Robots: Sensing, Control, Decision-Making, and Applications*, chapter Behavior-Based Coordination in Multi-Robot Systems. Marcel Dekker, Inc, 2005.
- [KH04] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2149–2154, Sendai, Japan, Sep 2004.
- [Kor09] Broadmann Korbinian. *Vergleichende Lokalisationstheorie der Groshirnrinde*. Leipzig, 1909.

- [LDW92] John J. Leonard and Hugh F. Durrant-Whyte. *Directed Sonar Sensing for Mobile Robot Navigation*. Kluwer Academic Press, Boston, MA, 1992.
- [LR00] Giuseppe Luppino and Giacomo Rizzolatti. The organization of the frontal motor cortex. *News in Physiological Sciences*, 15:219–224, October 2000.
- [MFF⁺97] Akira Murata, Luciano Fadiga, Leonardo Fogassi, Vittorio Gallese, Vassilis Raos, and Giacomo Rizzolatti. Object representation in the ventral premotor cortex (area f5) of the monkey. *Journal of Neurophysiology*, 78:2226–2230, October 1997.
- [Mic] Sun Microsystems. <http://www.sun.com/>.
- [ML01] Massimo Matelli and Giuseppe Luppino. Parietofrontal circuits for action and space perception in the macaque monkey. *Neuroimage*, 14:S27–S32, 2001.
- [MLR91] Massimo Matelli, Giuseppe Luppino, and Giacomo Rizzolatti. Architecture of superior and mesial area 6 and the adjacent cingulate cortex in the macaque. *Journal of Comparative Neurology*, 311(4):445–462, 1991.
- [OA03] Erhan Oztop and Michael A. Arbib. Schema design and implementation of the grasp-related mirror

- neuron system. *Biological Cybernetics*, 87:116–140, August 2003.
- [Pea95] Barak A. Pearlmutter. Gradient calculations for dynamic recurrent neural networks: A survey. *IEEE Transactions on Neural Networks*, 6(5):1212–1228, September 1995.
- [Pin87] Fernando J. Pineda. Generalization of back propagation to recurrent and higher order neural networks. In *NIPS*, pages 602–611, 1987.
- [PSM05] Roberto Prevede, Matteo Santoro, and Francesco Mariotti. A biologically inspired visuo-motor control model based on a deflationary interpretation of mirror neurons. In *COGSCI05 (XXVII Annual Meeting of the Cognitive Science Society)*, July 2005.
- [PT04a] Rainer W. Paine and Jun Tani. Evolved motor primitives and sequences in a hierarchical recurrent neural network. In *GECCO (1)*, pages 603–614, 2004.
- [PT04b] Rainer W. Paine and Jun Tani. Motor primitive and sequence self-organization in a hierarchical recurrent neural network. *Neural Networks*, 17:1291–1309, 2004.

- [RA98] Giacomo Rizzolatti and Michael A. Arbib. Language within our grasp. *Trends in Neurosciences*, 21:188–194, May 1998.
- [RL01] Giacomo Rizzolatti and Giuseppe Luppino. The cortical motor system. *Neuron*, 31:889–901, September 2001.
- [RLM98] Giacomo Rizzolatti, Giuseppe Luppino, and Massimo Matelli. The organization of the cortical motor system: new concepts. *Electroencephalography and Clinical Neurophysiology*, 106:283–296, April 1998.
- [RN95] Stuart Russel and Peter Norvig. *Artificial Intelligence: a modern approach*. Prentice Hall, New York, 1995.
- [TMGMA90] Masato Taira, Seiichiro Mine, A. P. Georgopoulos, and Sakata Hideo Murata Akira. Parietal cortex neurons of the monkey related to the visual guidance of hand movement. *Experimental Brain Research*, 83(1):29–36, December 1990.
- [UM82] Leslie G. Ungerleider and Mortimer Mishkin. *Two Cortcal Visual Systems*, pages 549–586. The MIT Press, 1982.

- [YB94] Brian M. Yamauchi and Randall D. Beer. Sequential behavior and learning in evolved dynamical neural networks. *Adaptive Behavior*, 2(3):219–246, 1994.